

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Частотные методы

Отчёт по лабораторной работе №5

Работу
выполнил:
Сухов Н. М.
Группа: R3235
Преподаватель:
Догадин Е. В.

Санкт-Петербург
2025

Содержание

1. Задание 1. Непрерывное и дискретное преобразование Фурье	3
1.1. Численное интегрирование	4
1.2. Использование DFT	9
1.3. Объяснения	14
1.4. Приближение непрерывного с помощью DFT	15
2. Сэмплирование	19
3. Приложение	31

1. Задание 1. Непрерывное и дискретное преобразование Фурье

Прежде, чем приступить к выполнению задания, рассмотрим знакомую прямоугольную функцию $\Pi : \mathbb{R} \rightarrow \mathbb{R}$:

$$\Pi(t) = \begin{cases} 1, & |t| \leq 1/2, \\ 0, & |t| > 1/2 \end{cases}$$

Её Фурье-образом будет являться аналитическое выражение:

$$\hat{\Pi}(\nu) = \int_{-\infty}^{+\infty} \Pi(t) e^{-2\pi i \nu t} dt$$

Вычислим вручную аналитически образ Фурье функции $\Pi(t)$:

$$\hat{\Pi}(\nu) = \int_{-\infty}^{+\infty} \Pi(t) e^{-2\pi i \nu t} dt = \int_{-0.5}^{0.5} e^{-2\pi i \nu t} dt = \int_{-0.5}^{0.5} \frac{1}{-2\pi i \nu} d e^{-2\pi i \nu t} = \frac{e^{-2\pi i \nu t}}{-2\pi i \nu} \Big|_{-0.5}^{0.5} = \frac{\sin(\pi \nu)}{\pi \nu} = \text{sinc}(\pi \nu)$$

Построим графики функций $\Pi(t)$ (см. рис. 1.1) и $\hat{\Pi}(\nu)$ (см. рис. 1.2)

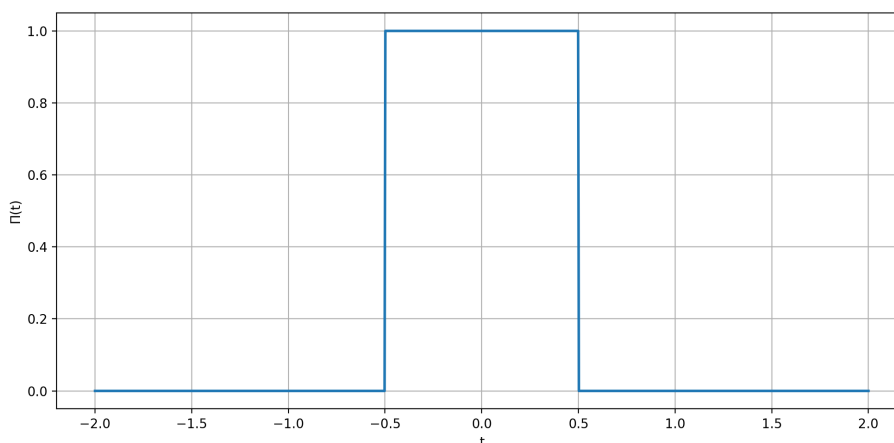


Рисунок 1.1. График $\Pi(t)$

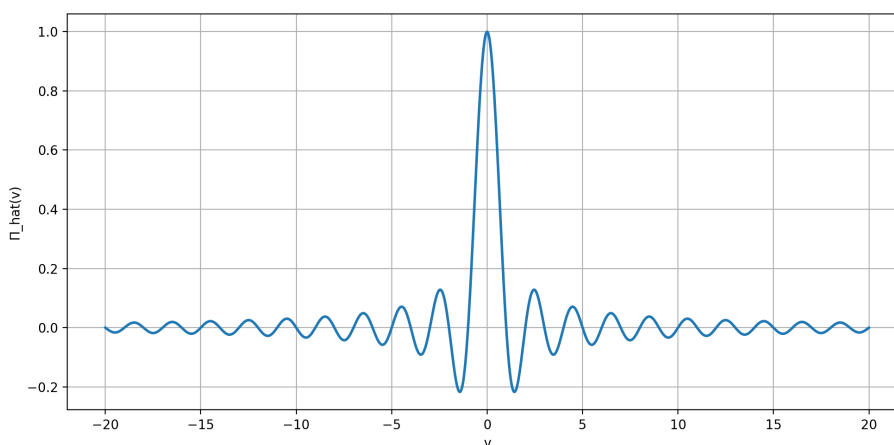


Рисунок 1.2. График $\hat{\Pi}(\nu)$

1.1. Численное интегрирование

Найдём Фурье-образ функции $\Pi(t)$ с помощью численного интегрирования (функция `trapz`).

Метод трапеций для численного приближённого вычисления определённого интеграла от функции $f(x)$ на интервале $[a, b]$ выглядит так:

$$S = \sum_{i=1}^{N-1} \frac{f(x_i) + f(x_{i-1})}{2} \cdot \Delta x, \quad \Delta x = x_i - x_{i-1}, \quad N = \frac{b - a}{\Delta x}$$

Применять метод будем для вычисления Фурье от наших функций ($-\infty$ и ∞ заменятся на пределы численного интегрирования):

$$\begin{cases} \Pi(t) = \int_{-\infty}^{\infty} \hat{\Pi}(\nu) e^{2\pi i \nu t} d\nu \\ \hat{\Pi}(\nu) = \int_{-\infty}^{\infty} \Pi(t) e^{-2\pi i \nu t} dt \end{cases}$$

Исследуем влияние параметров T (промежутков интегрирования по времени) и Δt (шаг дискретизации) на получаемый путём численного интегрирования Фурье-образ функции (см. рис. 1.3) и восстановленную функцию из образа (см. рис. 1.4), зафиксируем $V, \Delta \nu$.

Исследуем влияние параметров V (промежутков интегрирования по времени) и $\Delta \nu$ (шаг дискретизации) на получаемый путём численного интегрирования Фурье-образ функции (см. рис. 1.5) и восстановленную функцию из образа (см. рис. 1.6), зафиксируем $T, \Delta t$.

Анализ и выводы

Видим, что уменьшение шага интегрирования по времени даёт более точные результаты при вычислении Фурье-образа, сильно увеличивается период в Фурье-образе, за счет чего пропадают наблюдаемые пики вблизи главных частот, и при восстановлении сигнал не имеет сильного искажения. Увеличение периода T приводит к появлению наблюдаемой периодичности в восстановленном сигнале, при этом не имеет большого влияния на форму самого восстановленного сигнала. Уменьшение шага интегрирования по частоте приводит к более точной аппроксимации Фурье-образа, но не оказывает влияния на форму восстановленного сигнала (если только не брать слишком большие значения, при которых вся важная информация просто потеряется). При увеличении V сигнал восстанавливается лучше, можем наблюдать эффект Гиббса. При правильно подобранных параметрах $\Delta \nu, \Delta t, V, T$ метод работает с хорошей точностью, но вычисления занимают много времени.

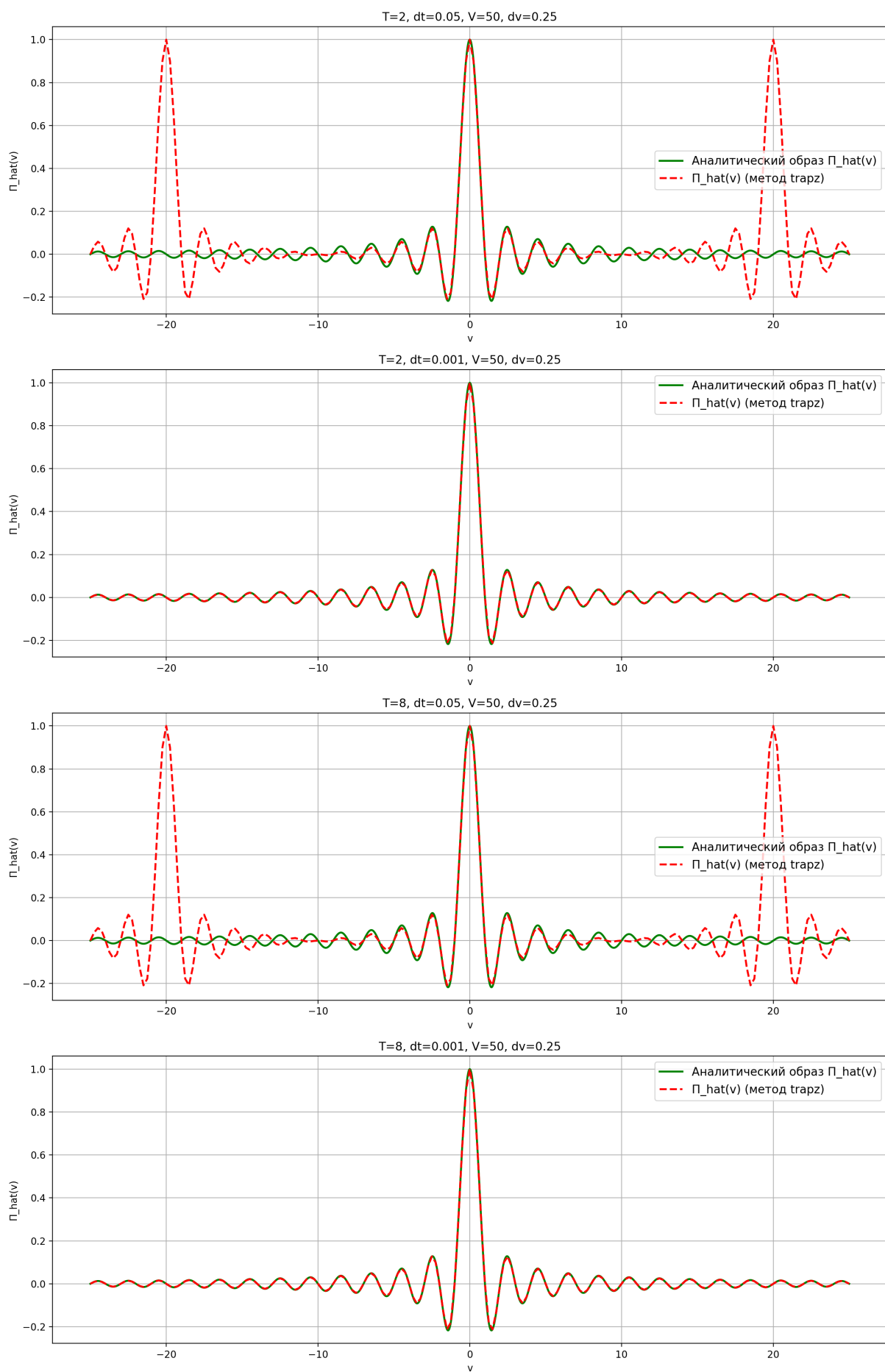


Рисунок 1.3. Сравнение графиков $\hat{\Pi}(\nu)$

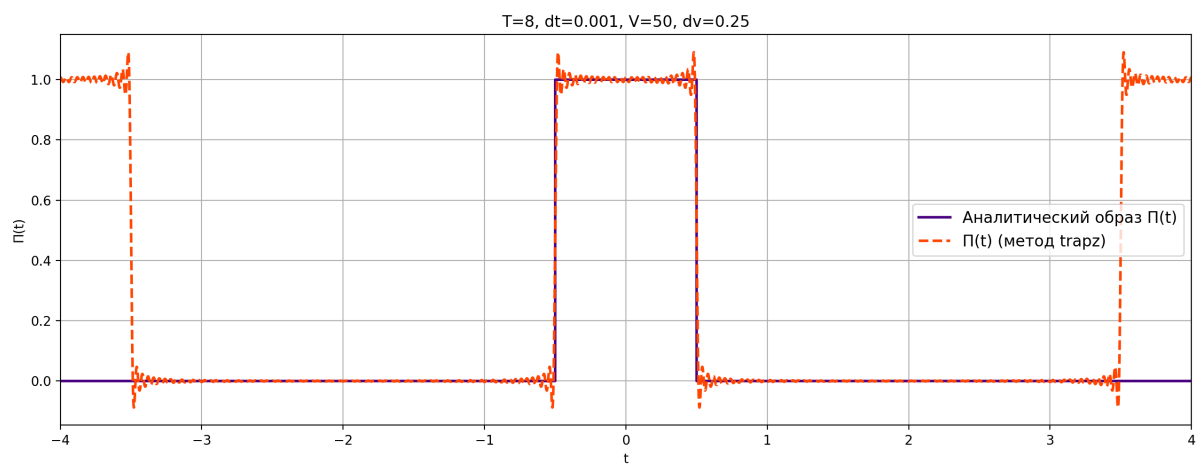
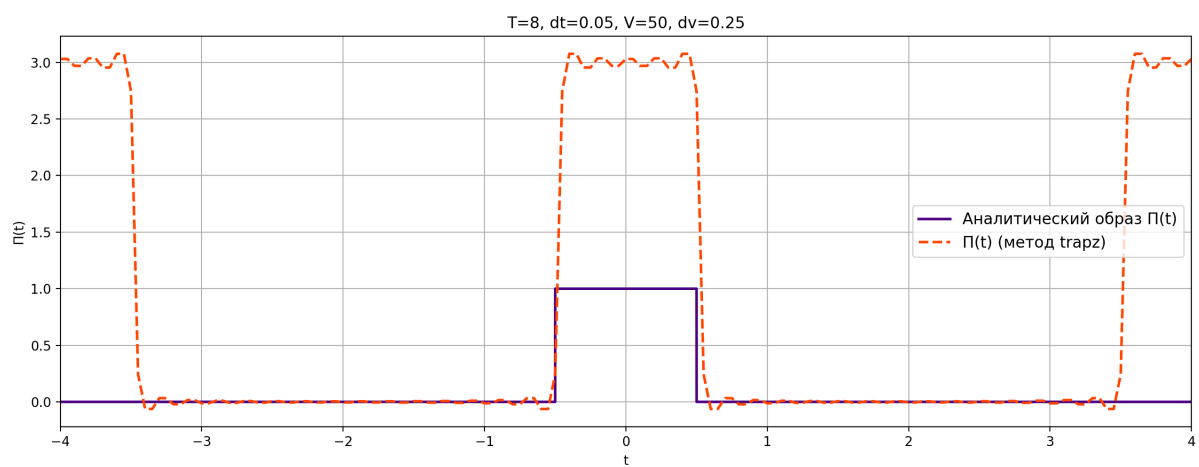
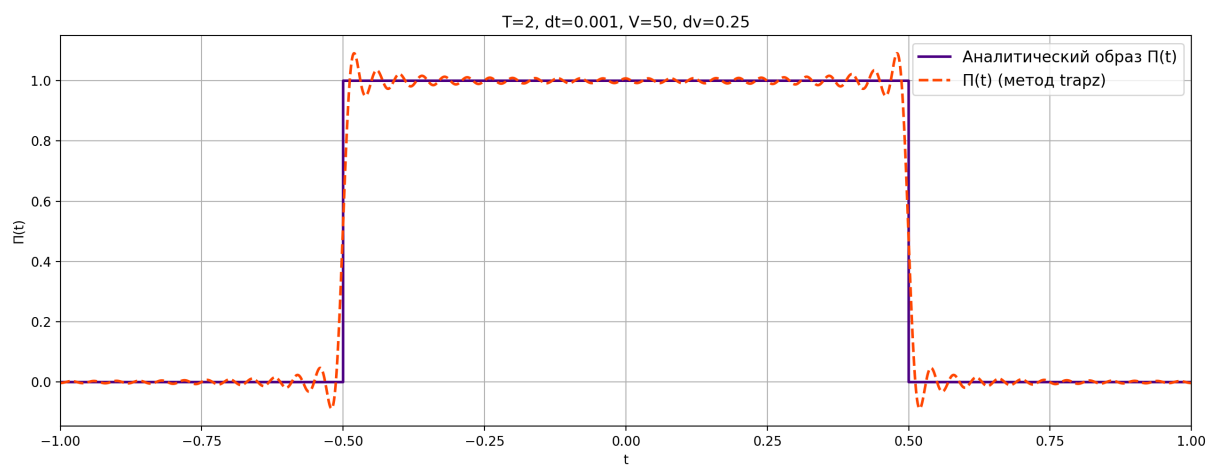
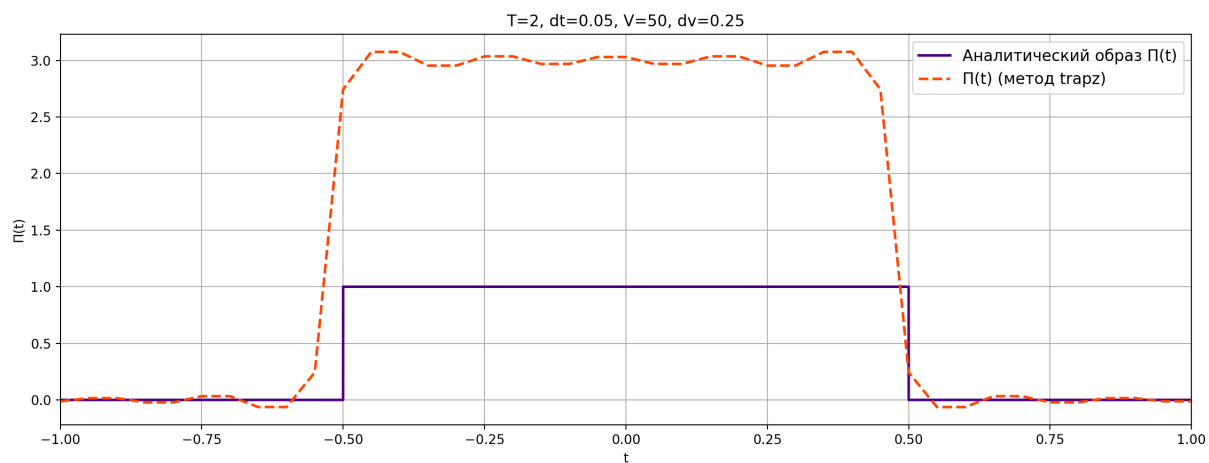


Рисунок 1.4. Сравнение графиков $\Pi(t)$

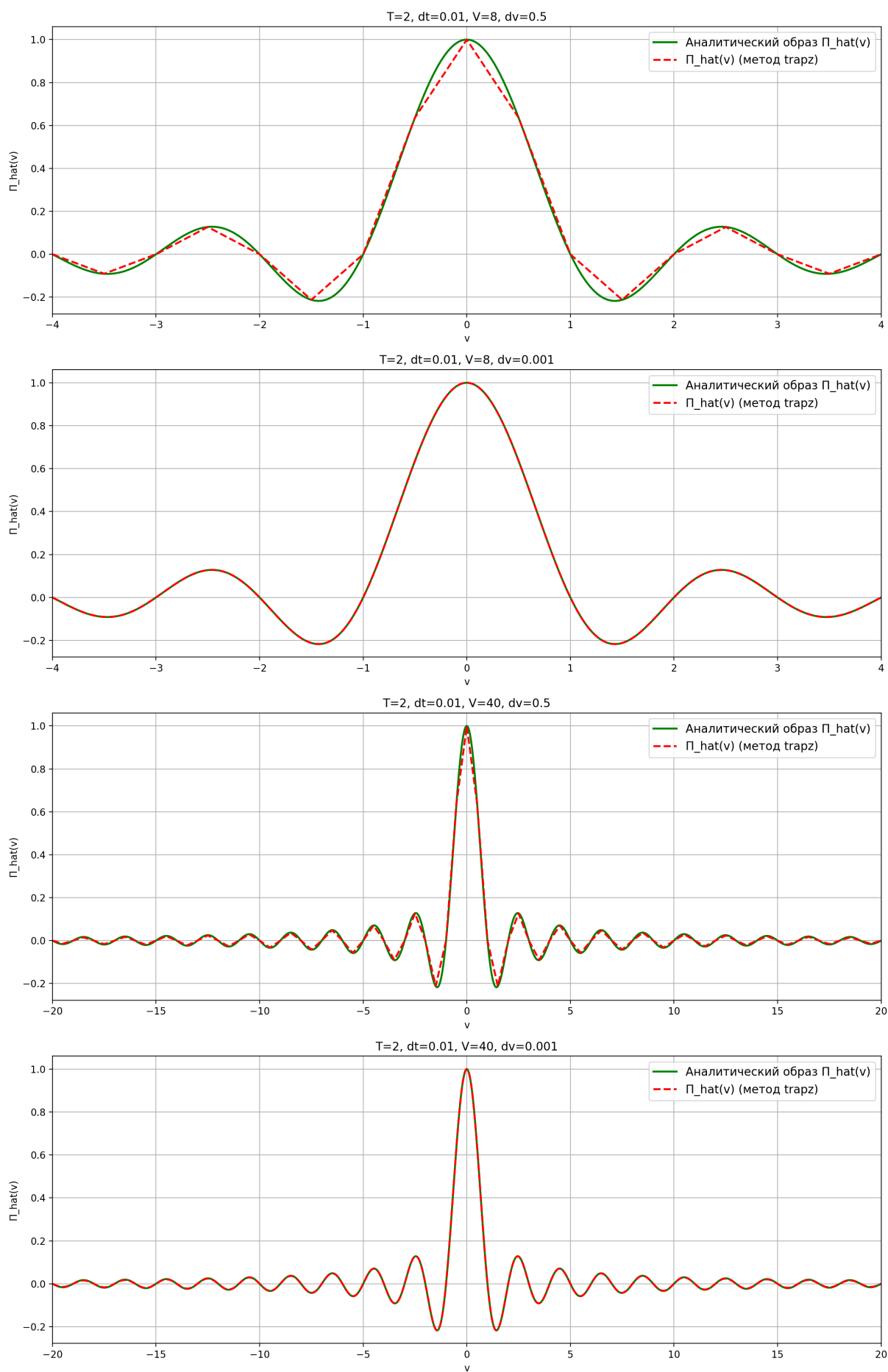


Рисунок 1.5. Сравнение графиков $\hat{\Pi}(\nu)$

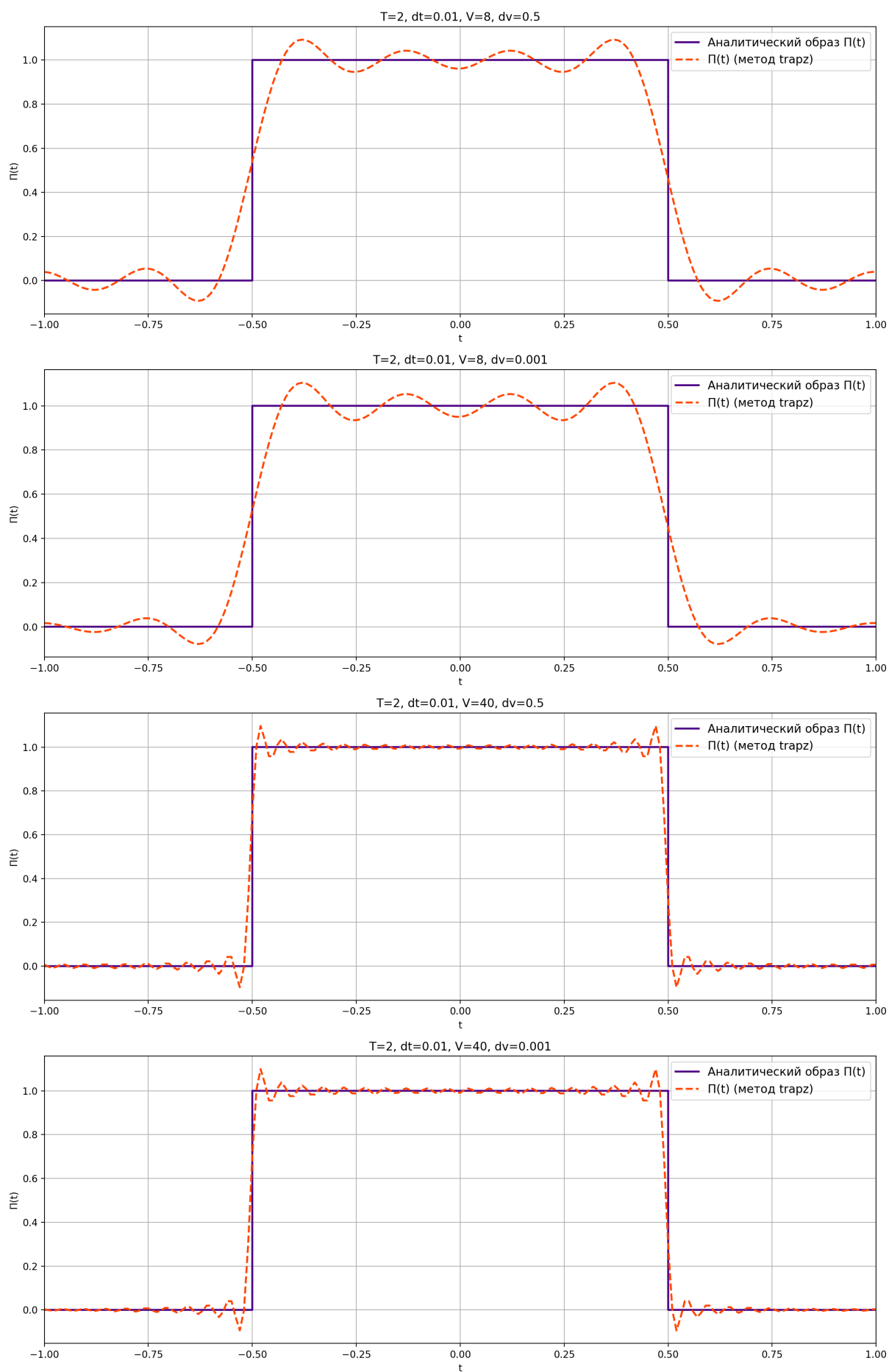


Рисунок 1.6. Сравнение графиков $\Pi(t)$

1.2. Использование DFT

Найдём Фурье-образ функции $\Pi(t)$ с помощью дискретного преобразования Фурье, используя его так, чтобы преобразование было унитарным. Выполним обратное преобразование от найденного Фурье-образа с помощью обратного дискретного преобразования. Формулы прямого и обратного унитарного DFT :

$$\begin{cases} X_m = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x_n e^{-i2\pi(\frac{mn}{N})} \\ x_n = \frac{1}{\sqrt{N}} \sum_{m=0}^{N-1} X_m e^{i2\pi\frac{mn}{N}} \end{cases}$$

Связь параметров $\Delta\nu$ с T , и Δt с V выглядит так:

$$\Delta\nu = \frac{1}{T}, \quad \Delta t = \frac{1}{V}$$

Для вычислений будем использовать быстрое преобразование Фурье - алгоритм, позволяющий вычислять дискретное преобразование Фурье за $O(N \log(N))$ действий.

Исследуем влияние параметров T (промежуток интегрирования по времени) и Δt (шаг дискретизации) на получаемый при помощи `fft` Фурье-образ функции и восстановленную функцию из образа (см. рис. 1.7-1.10). Найдём соответствующие им параметры V , $\Delta\nu$.

Анализ и выводы

Уменьшение шага дискретизации даёт больший охват спектра частот, что даёт более точное описание сигнала, восстановления происходит точнее. Увеличение периода T также повышает точность восстановления сигнала благодаря тому, что уменьшается шаг дискретизации в частотной области и спектр частот сигнала описывается более точно. При этом, Фурье-образ не сходится с вычисленным аналитически, потому что дискретное преобразование Фурье предусмотрено для дискретных непериодических функций. Так же при использовании DFT получается избежать эффекта Гиббса, и в целом восстановленный сигнал при правильно подобранных параметрах намного лучше совпадает с исходным. Работает данный методкратно быстрее численного интегрирования из предыдущего пункта.

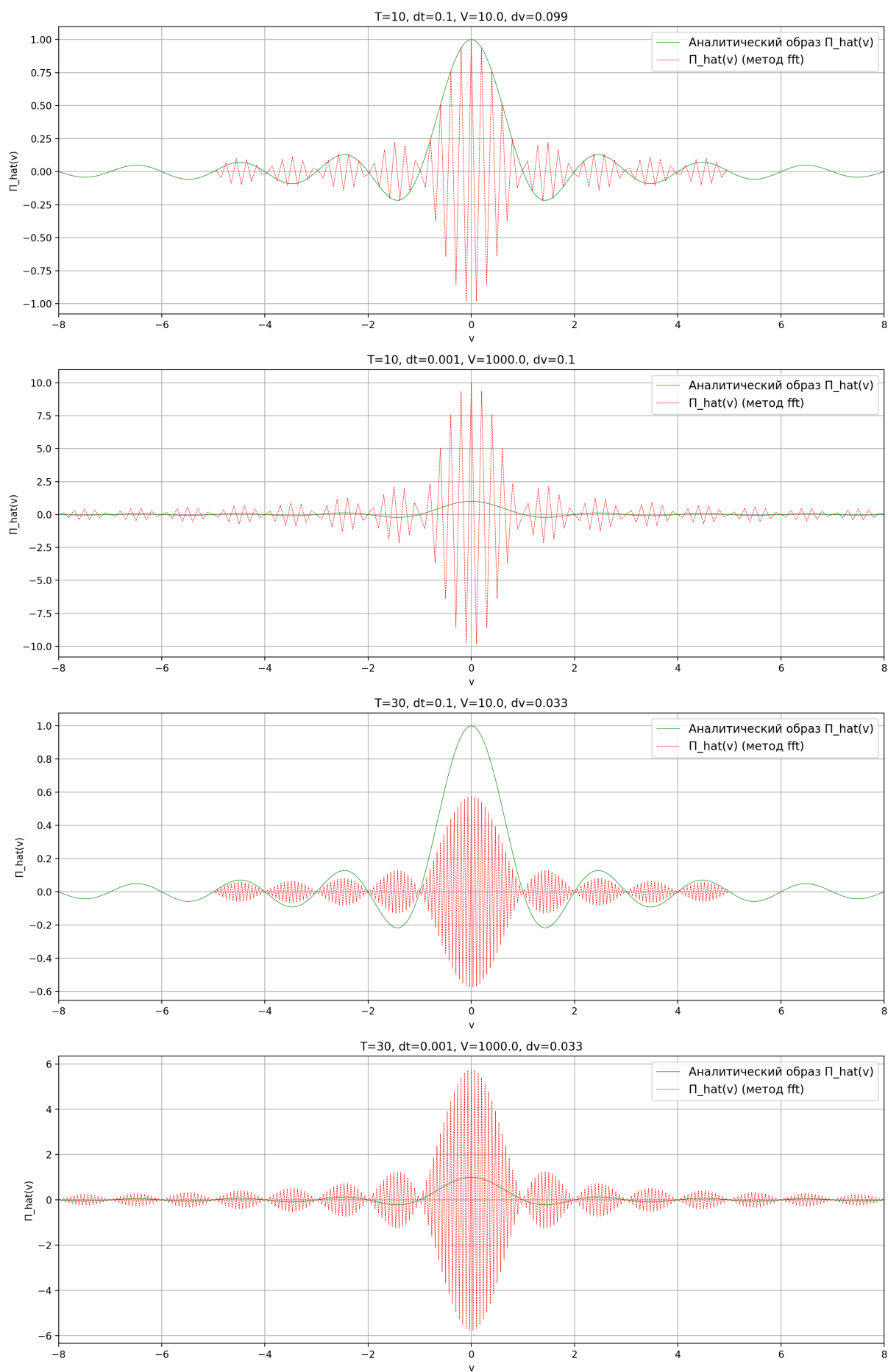


Рисунок 1.7. Сравнение графиков $\hat{\Pi}(\nu)$

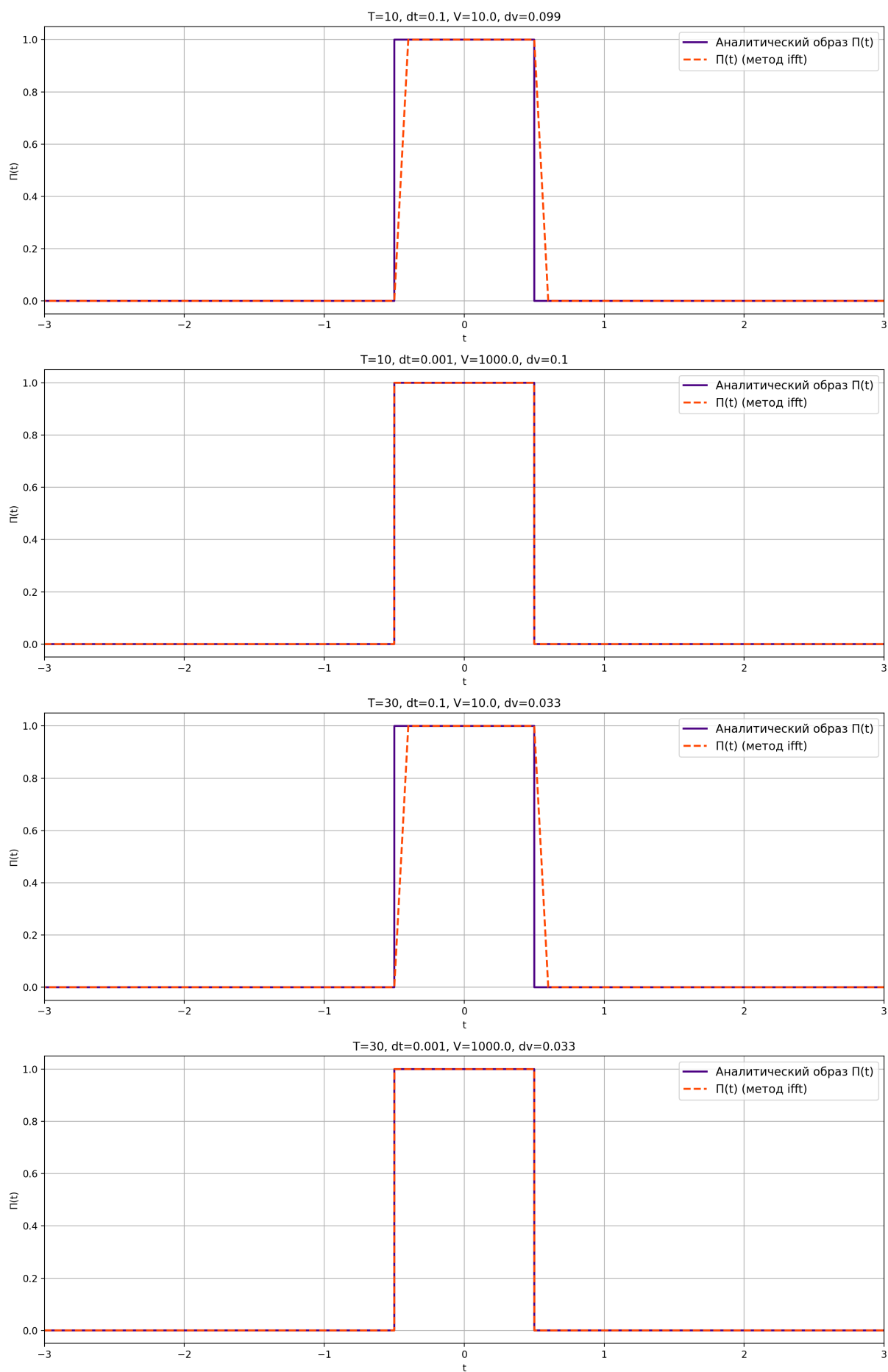


Рисунок 1.8. Сравнение графиков $\Pi(t)$

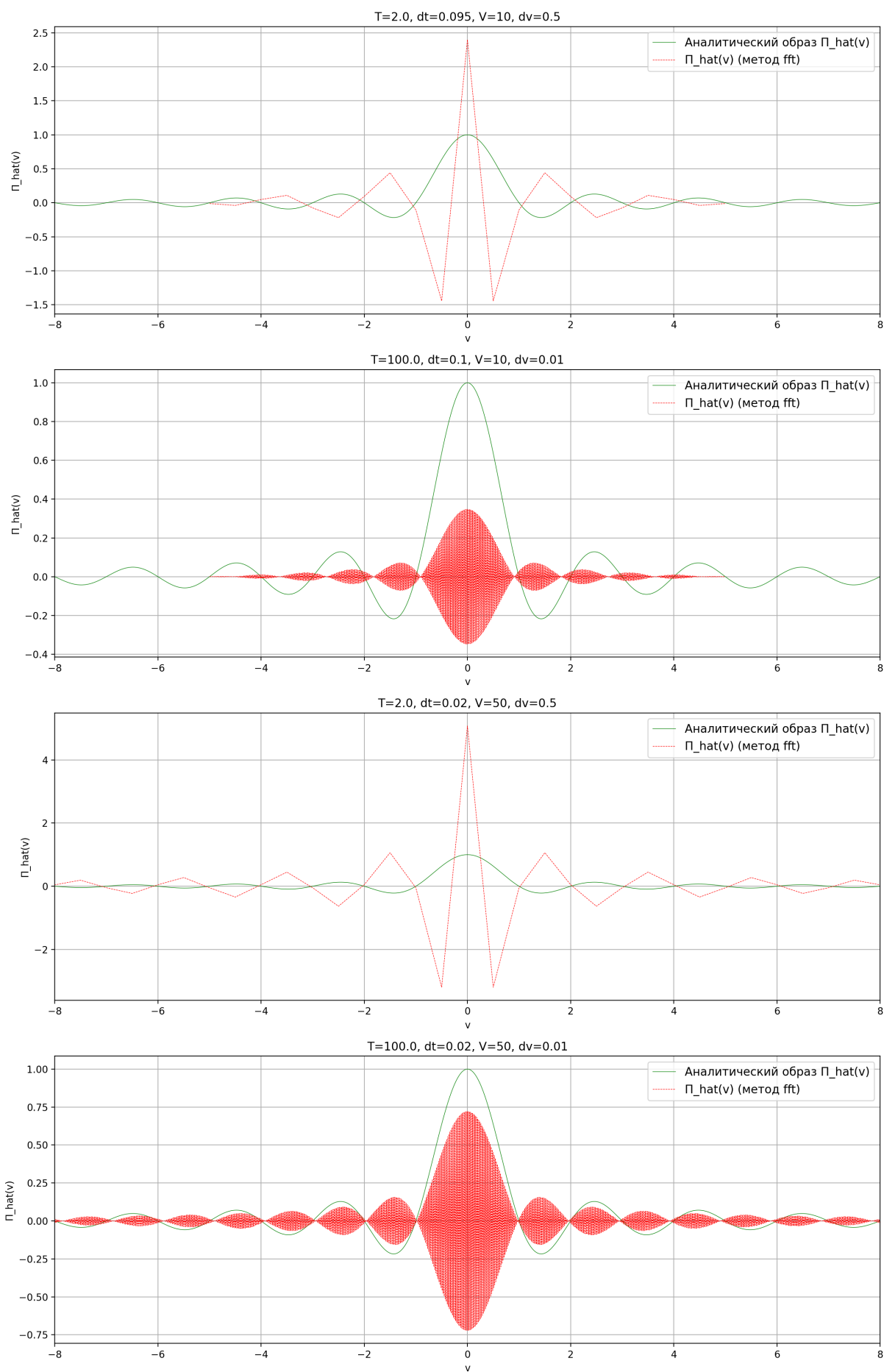


Рисунок 1.9. Сравнение графиков $\hat{\Pi}(\nu)$

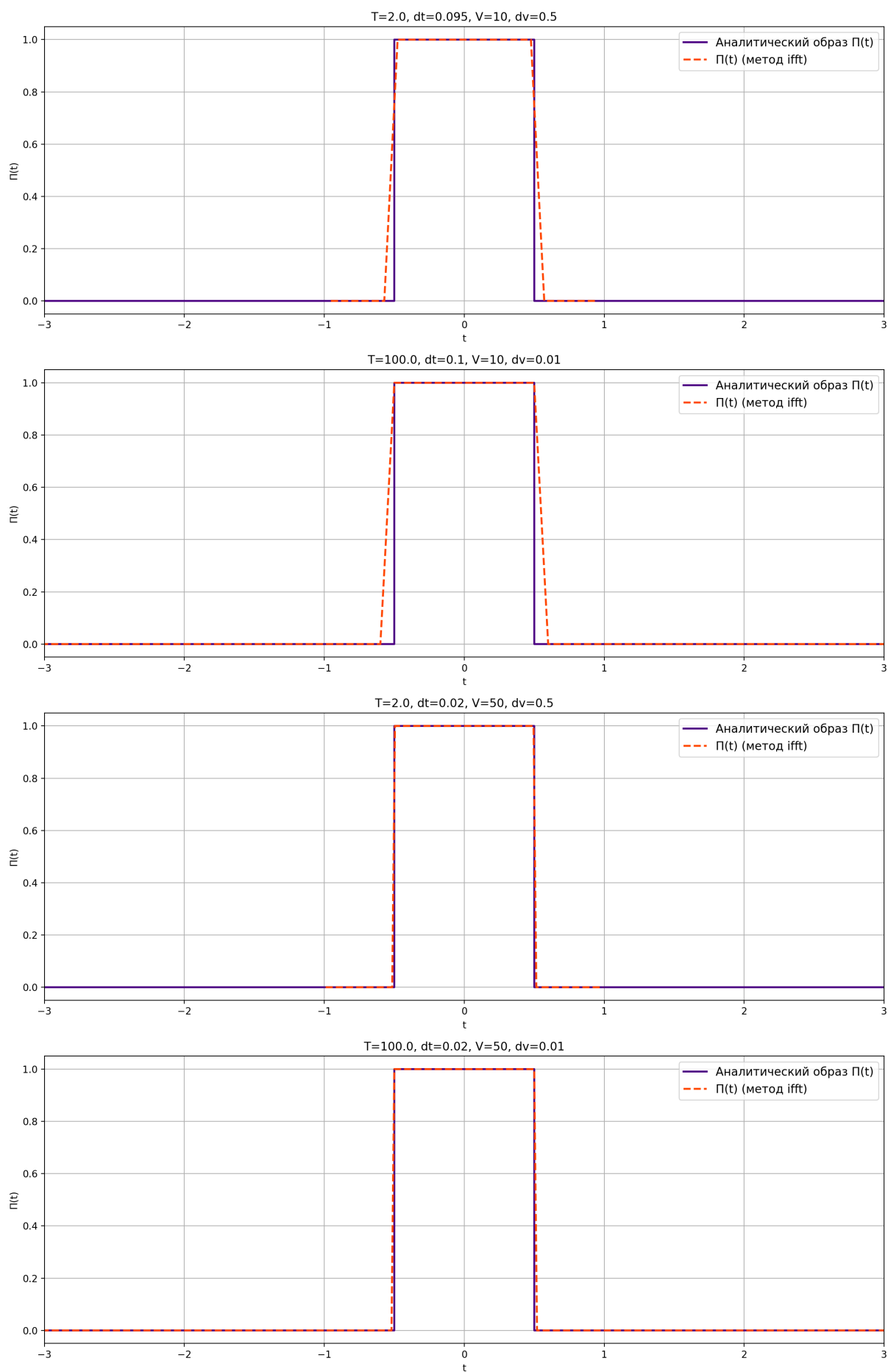


Рисунок 1.10. Сравнение графиков $\Pi(t)$

1.3. Объяснения

Проблема Фурье-образа

Метод `trapz` фактически аппроксимирует вычисление аналитически найденного интеграла, поэтому при увеличении T и устремлении Δt к нулю мы получаем истинный Фурье-образ (с некоторой погрешностью). DFT же должен работать с дискретными и конечными, а не непрерывными функциями, поэтому при нахождении Фурье-образа от непрерывной функции, заданной на бесконечности, мы не получим истинного Фурье-образа. Однако, в силу обратимости преобразований DFT и обратного DFT, мы можем восстановить исходную функцию корректно.

Точность обратного преобразования

Метод `trapz` справляется с восстановлением исходной функции хуже DFT, потому что у прямоугольной функции есть скачки, из-за которых возникает эффект Гиббса в восстановленной функции. DFT справляется с функцией благодаря обратимости преобразования.

Скорость

Для вычисления интеграла методом трапеций по N точкам необходимо произвести суммирование N раз. Соответственно, чтобы получить N значений функции, нам понадобится N^2 операций, время работы алгоритма в Big O нотации равно $O(N^2)$. FFT же является алгоритмом для ускоренного вычисления DFT, основанным на использовании симметрии матрицы преобразования Фурье, который работает за время $O(N \log(N))$. Наглядное сравнение времени работы алгоритмов при увеличении количества данных представлено на рисунке 1.11.

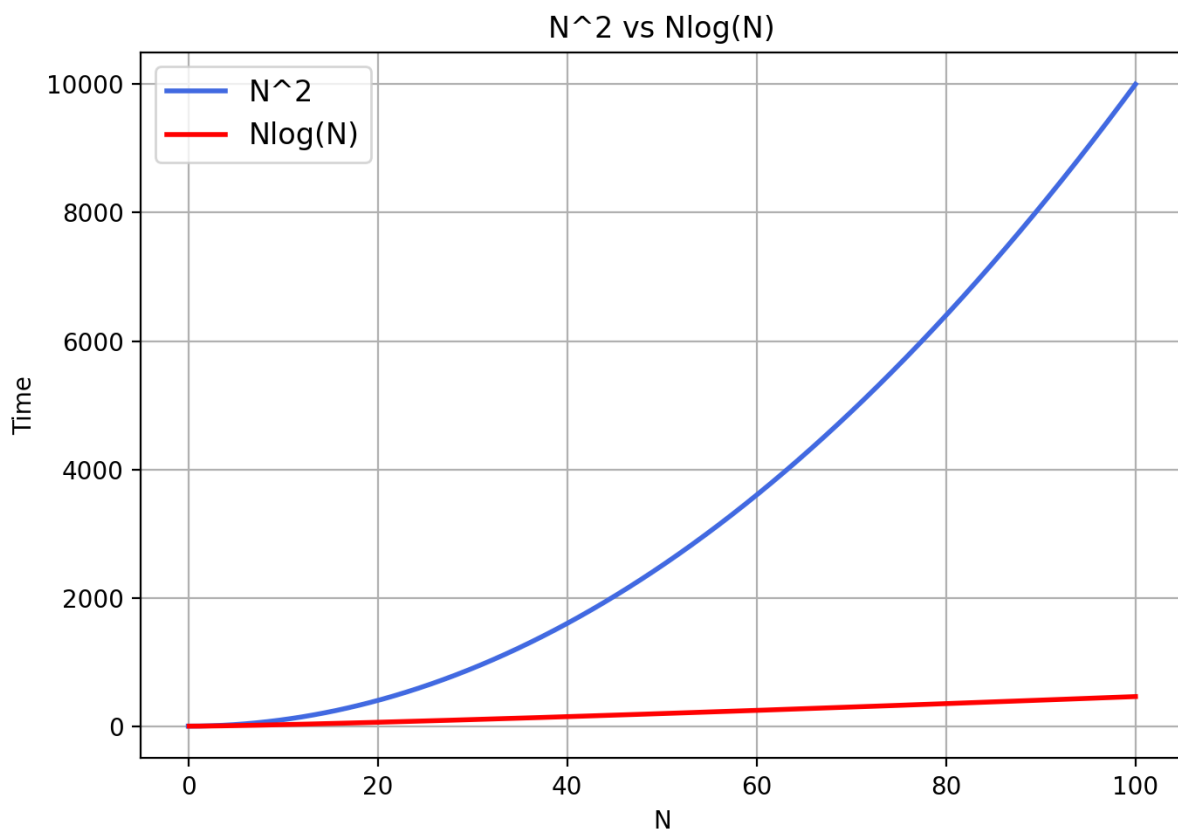


Рисунок 1.11. Сравнение времени работы алгоритмов

1.4. Приближение непрерывного с помощью DFT

Найдём способ получить правильный Фурье-образ, соответствующий непрерывному преобразованию Фурье, используя функцию `fft` и не прибегая к численному интегрированию. Также найдем способ восстановить исходный сигнал по полученному Фурье-образу с помощью `ifft`.

Воспользуемся суммой Римана:

$$\int_a^b f(t)dt \approx \sum_{n=0}^{N-1} f(t_n)\Delta t, \quad \text{где } t_n = a + n\Delta t, \quad \Delta t = \frac{b-a}{N-1}$$

Аппроксимируем преобразование Фурье суммой Римана. Это можно сделать так:

$$\hat{f}(\nu) = \int_{-T/2}^{T/2} f(t)e^{-2\pi i \nu t} dt \approx \sum_{n=0}^{N-1} f(t_n)e^{-2\pi i \nu t_n} \Delta t, \quad \text{где } t_n = -\frac{T}{2} + n\Delta t,$$
$$\Delta t = \frac{T}{N-1}, \quad \nu = \frac{m}{N\Delta t}$$

Или так:

$$\hat{f}(\nu) \approx \int_{-T/2}^{T/2} f(t)e^{-2\pi i \nu t} dt \rightarrow \hat{f}(\nu_m) \approx c_m \sum_{n=0}^{N-1} f(t_n)e^{-2\pi i \frac{nm}{N}}$$

Отсюда найдём коэффициент c_m :

$$\sum_{n=0}^{N-1} f(t_n)e^{-2\pi i \nu t_n} \Delta t = c_m \sum_{n=0}^{N-1} f(t_n)e^{-2\pi i \frac{nm}{N}}$$
$$c_m = \Delta t \cdot e^{-2\pi i (\nu t_n - \frac{mn}{N})} = \Delta t \cdot e^{-2\pi i (\frac{m}{N\Delta t} (n\Delta t - \frac{T}{2}) - \frac{mn}{N})} = \Delta t \cdot e^{2\pi i \nu \frac{T}{2}}$$

Или, если записать в более общем виде:

$$c_m = \Delta t \cdot e^{-2\pi i \nu t_0}, \quad \text{где } \nu = \frac{m}{N\Delta t}$$

Теперь, чтобы получить Фурье-образ, найдём коэффициенты c_m для всех частот и поэлементно домножим на массив точек, полученный из `fft` выполненного для функции $\Pi(t)$. И найдем приближение исходной функции, поделив массив точек из `ifft` на тот же коэффициент. Посмотрим на графики и убедимся, что данный алгоритм сработал (1.12-1.13).

Суть метода и почему он работает

Основная идея заключается в том, что мы представляем аппроксимацию непрерывного интеграла в виде дискретной суммы домноженной на поправочный коэффициент. Сумма Римана вида $\sum_{n=0}^{N-1} f(t_n)e^{-2\pi i \frac{nm}{N}}$ является естественным видом DFT, а коэффициент c_m решает следующие 2 проблемы, чтобы привести общее выражение к виду непрерывного преобразования Фурье, приближенного суммой Римана:

1. **Масштабирует DFT**, учитывая Δt для аппроксимации непрерывного интеграла дискретной суммой.

2. **Компенсирует фазовый сдвиг** на $\frac{T}{2}$ - поскольку DFT рассчитывает на начало сигнала в $t = 0$ по умолчанию, наш множитель $e^{-2\pi i \nu t_0}$ призван компенсировать такое поведение, сдвигая сигнал во временной области на эту разницу в половину периода.

Так же сравним с остальными методами (см. рис. 1.14-1.15).

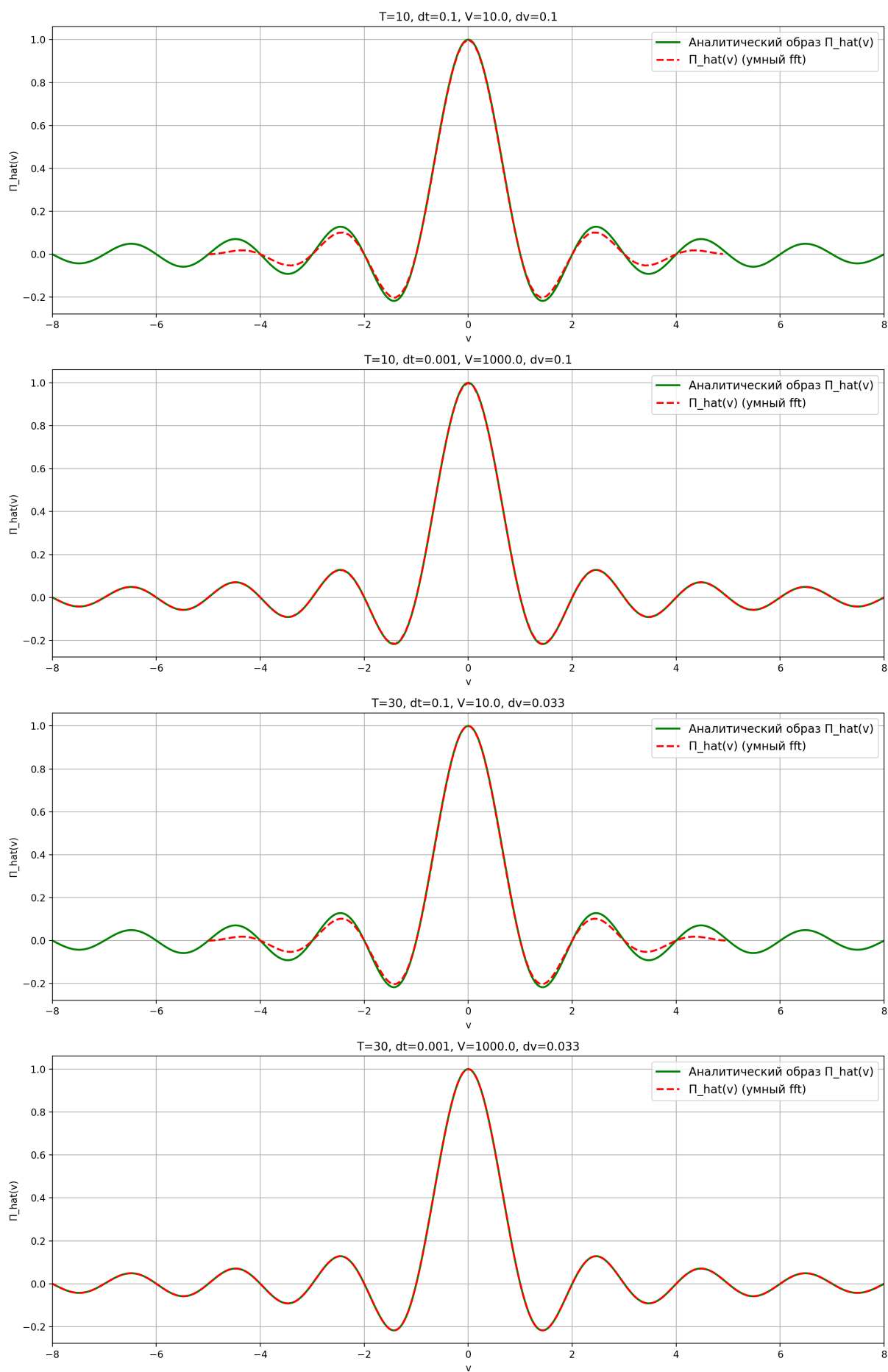


Рисунок 1.12. Применение умного fft

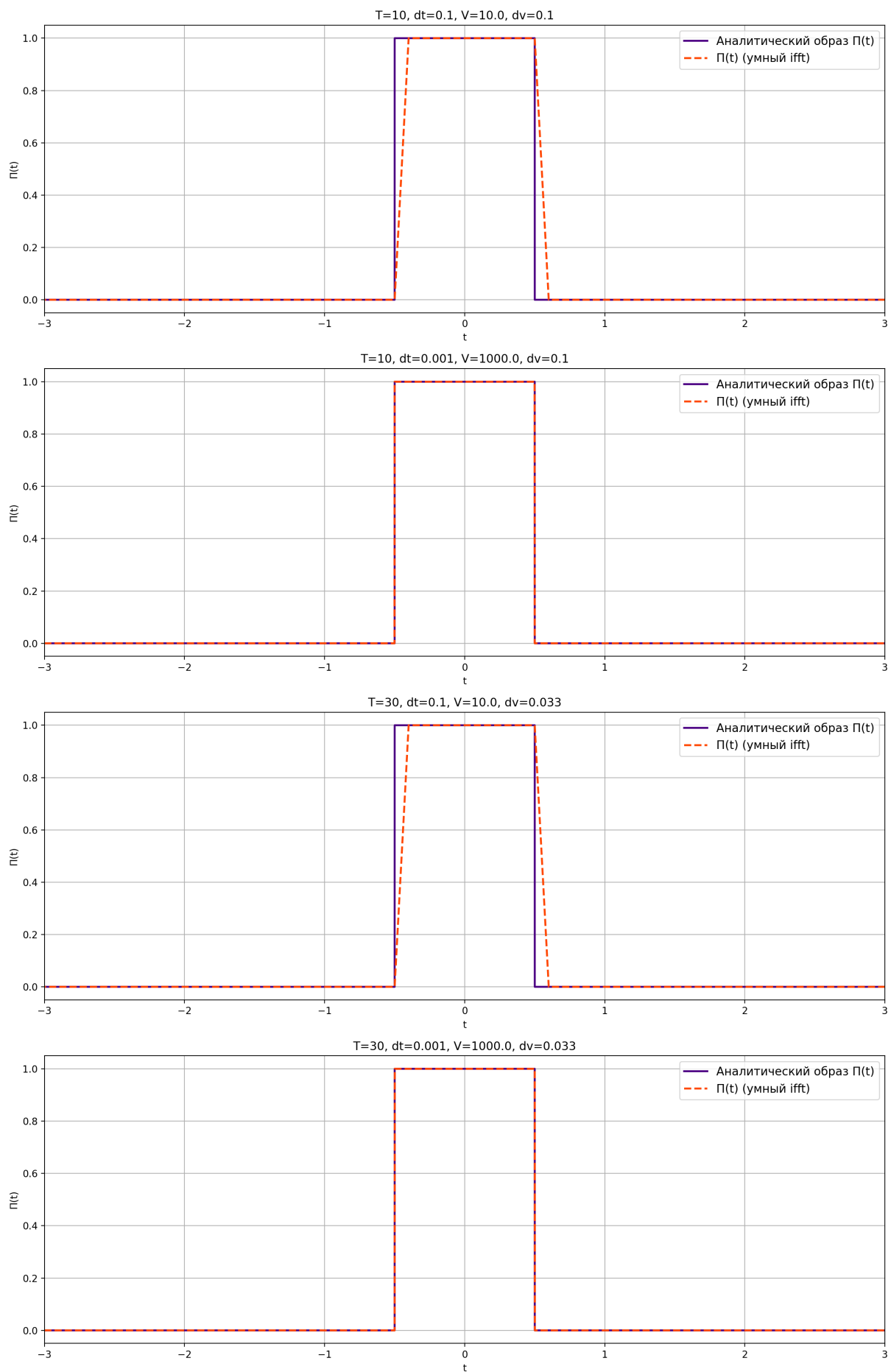


Рисунок 1.13. Применение умного ifft

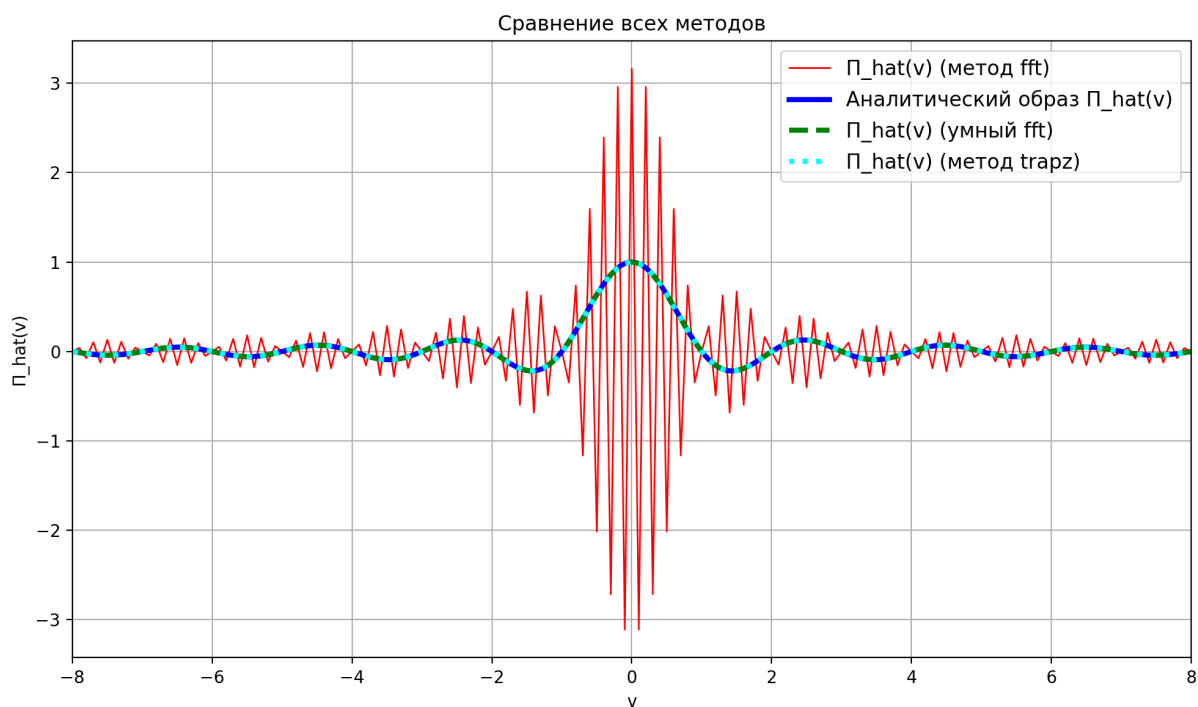


Рисунок 1.14. Сравнение умного fft с trapez и чистым fft

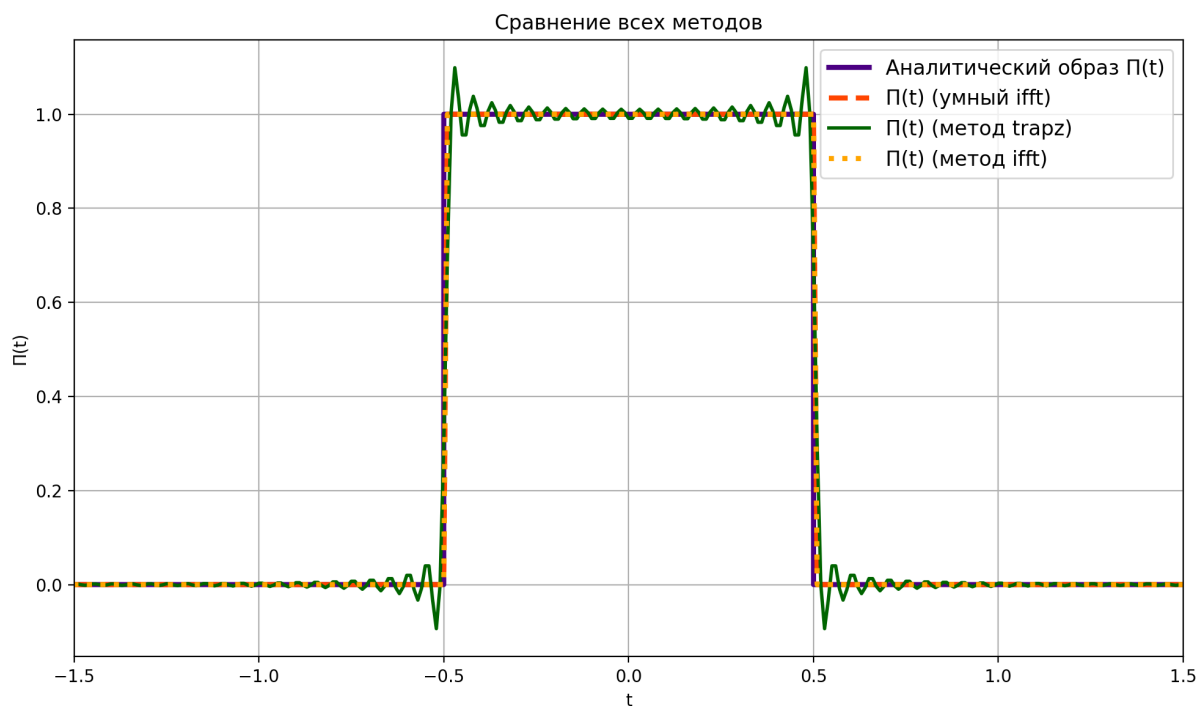


Рисунок 1.15. Сравнение умного ifft с trapez и чистым ifft

Метод умного использования FFT выигрывает по всем параметрам (в сумме) у остальных методов. Разве что, по скорости он может быть немного медленнее, чем DFT, потому что выполняет больше операций при вычислениях, но эта разница довольно мала, чтобы говорить здесь о каком-то преимуществе DFT. В остальном, он совмещает в себе точность DFT при восстановлении сигнала и корректное нахождение Фурье-образа, как у метода trapez.

Что касемо влияния параметров $\Delta t, T, \Delta \nu, V$, результат аналогичен обычному DFT - для достижения большей точности при восстановлении сигнала нужно выбирать больший T и уменьшать Δt .

2. Сэмплирование

Зададимся параметрами $a_1 = 3, a_2 = 2, \omega_1 = 4, \omega_2 = 7, \phi_1 = \pi, \phi_2 = 1, b = 6$ и рассмотрим функции:

$$y_1(t) = a_1 \sin(\omega_1 t + \phi_1) + a_2 \sin(\omega_2 t + \phi_2), y_2(t) = \text{sinc}(bt)$$

Мы будем исследовать теорему Найквиста-Шеннона-Котельникова на двух примерах. Чтобы результат численного моделирования был максимально близок к математическому, будем задавать рассматриваемые функции на как можно большем промежутке, на картинках же для наглядности выведем только наиболее весомую часть функции. Для начала построим графики функций и посмотрим на них (рис. 2.1).

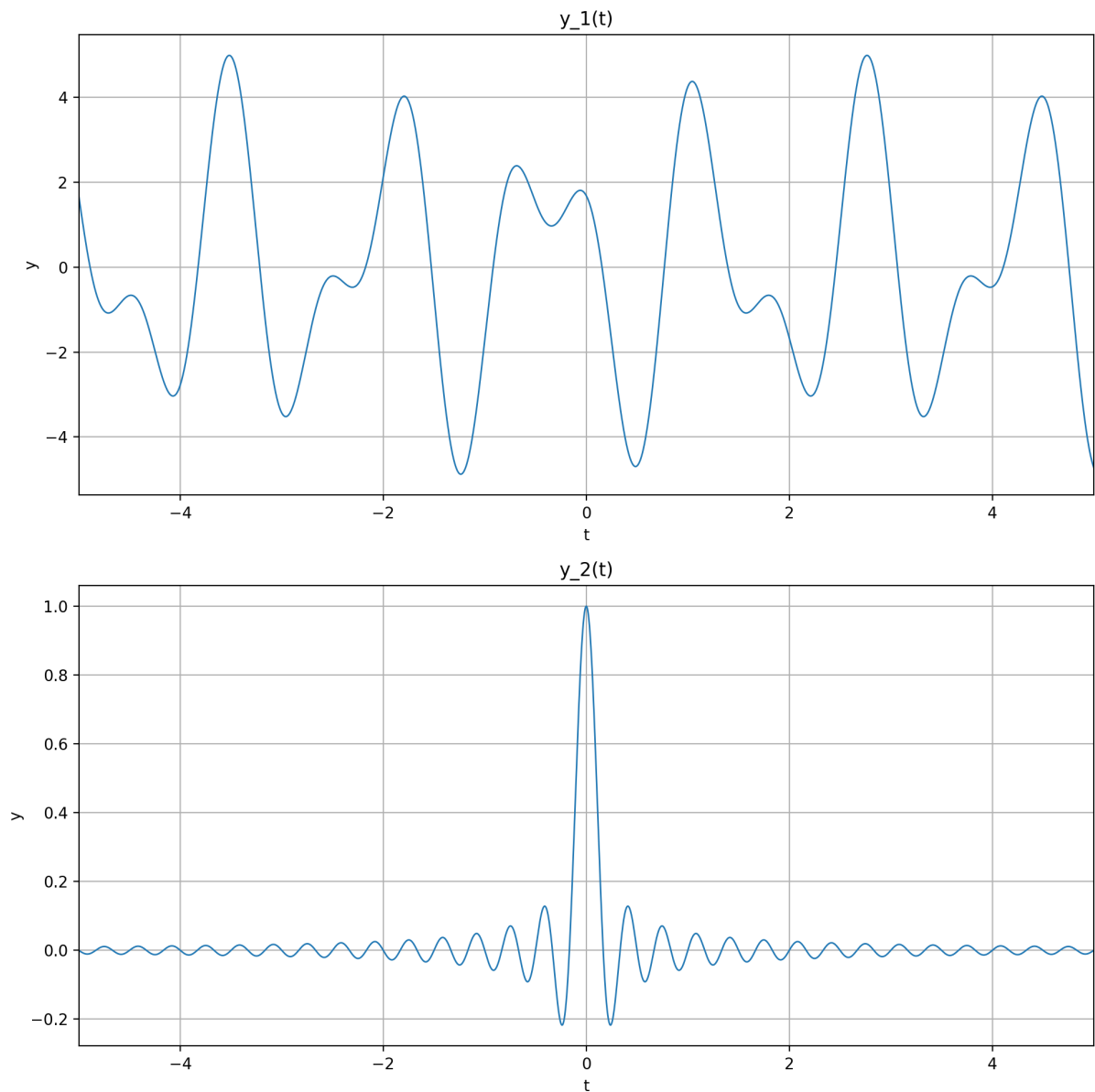


Рисунок 2.1. $y(t)$

Теперь зададим сэмплированный вариант указанных функций: рассмотрим разреженный вариант массива времени и соответствующий ему массив значений. Построим дискретный график поверх непрерывного для разных значений Δt (см. рис. 2.2-2.3).

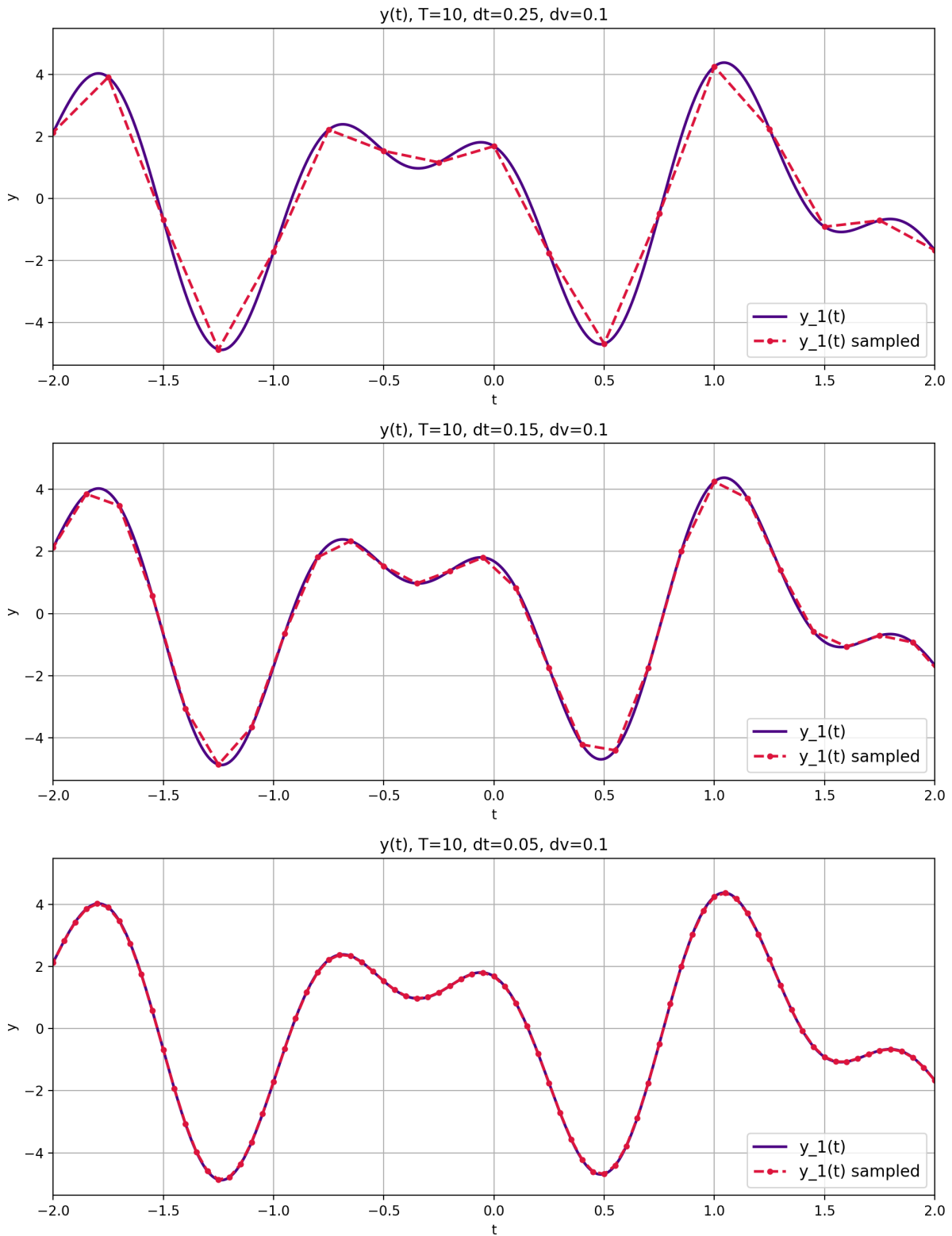


Рисунок 2.2. Сравнение непрерывного и сэмплированного варианта $y_1(t)$

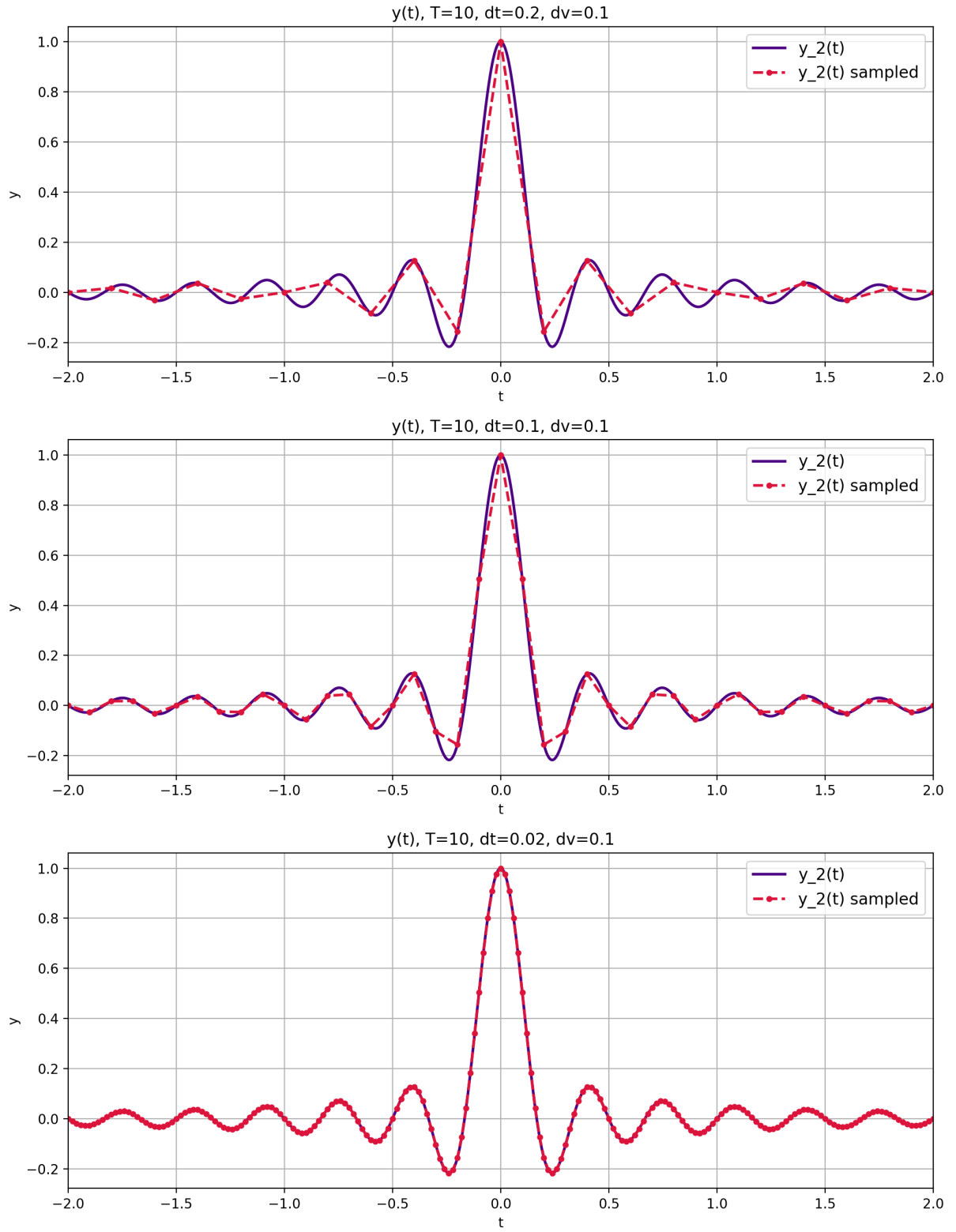


Рисунок 2.3. Сравнение непрерывного и сэмплированного варианта $y_2(t)$

Найдем параметр B для наших функций:

$$\hat{y}_1(\nu) = \frac{a_1}{2i}(\delta_{\frac{\omega_1}{2\pi}} - \delta_{-\frac{\omega_1}{2\pi}}) + \frac{a_2}{2i}(\delta_{\frac{\omega_2}{2\pi}} - \delta_{-\frac{\omega_2}{2\pi}}),$$

так как Фурье-образ синуса определяется выражением: $\mathcal{F}\{\sin(at)\}\mathcal{F}\{\frac{1}{2i}e^{iat} - \frac{1}{2i}e^{-iat}\} \simeq \{\frac{1}{2i}\delta_a - \frac{1}{2i}\delta_{-a}\}$

$$\hat{y}_1(\nu) = \frac{3}{2i}(\delta_{\frac{2}{\pi}} - \delta_{-\frac{2}{\pi}}) + \frac{1}{i}(\delta_{\frac{7}{2\pi}} - \delta_{-\frac{7}{2\pi}})$$

Получаем, что максимальная частота, присутствующая в y_1 равна $\frac{7}{2\pi} \Rightarrow B = \frac{7}{2\pi}$.

$$\hat{y}_2(\nu) = \mathcal{F}\{sinc(bt)\} = \frac{1}{|b|}\text{rect}\left(\frac{\nu}{b}\right) = \frac{1}{6}\text{rect}\left(\frac{\nu}{6}\right)$$

В нашем случае, прямоугольная функция не равна нулю в области $[-3, 3] \Rightarrow B = 3$.

Теорема Найквиста-Шеннона-Котельникова: Если Фурье-образ исходной функции содержится внутри отрезка $[-B, B]$, то функция может быть безошибочно восстановлена по дискретным данным, если шаг дискретизации $\Delta t < 1/(2B)$. Здесь B - это область определения $\hat{f}(\nu)$, сопоставляемая $f(t)$ при Фурье-преобразовании. Также верна следующая интерполяционная формула:

$$f(t) = \sum_{n=-\infty}^{\infty} f(t_n)\text{sinc}(2B(t - t_n)), t_n = \frac{n}{2B}$$

Применим интерполяционную формулу, и посмотрим как $\Delta t, T$ влияют на восстановление (см. рис. 2.4-2.7).

Мы видим, что как и предполагалось по теореме Найквиста-Шеннона-Котельникова, при $\Delta t < \frac{1}{2B}$ интерполированная функция полностью совпадает с исходной, при других значениях это не срабатывает. Так же можно сделать вывод, что смысла уменьшать шаг дискретизации после этого значения нет - только потратим время на вычисления.

Теперь найдём Фурье-образы функций при помощи умного `fft` для показательных значений шага дискретизации (см. рис. 2.8-2.11).

Отсюда видим, что период не влияет на качество интерполяции, но улучшает Фурье-образ сэмплированного сигнала.

Выводы

Полученные в данной лабораторной работе результаты согласуются с теоретическими представлениями. Мы исследовали разные подходы к реализации преобразования Фурье для практических вычислений, нашли их плюсы и недостатки (подробно описаны в предыдущих разделах). Узнали, как можно их совмещать, чтобы добиться лучшего результата (умный `fft`). Эксперимент с сэмплированием показал корректность теоремы Найквиста-Шеннона-Котельникова.

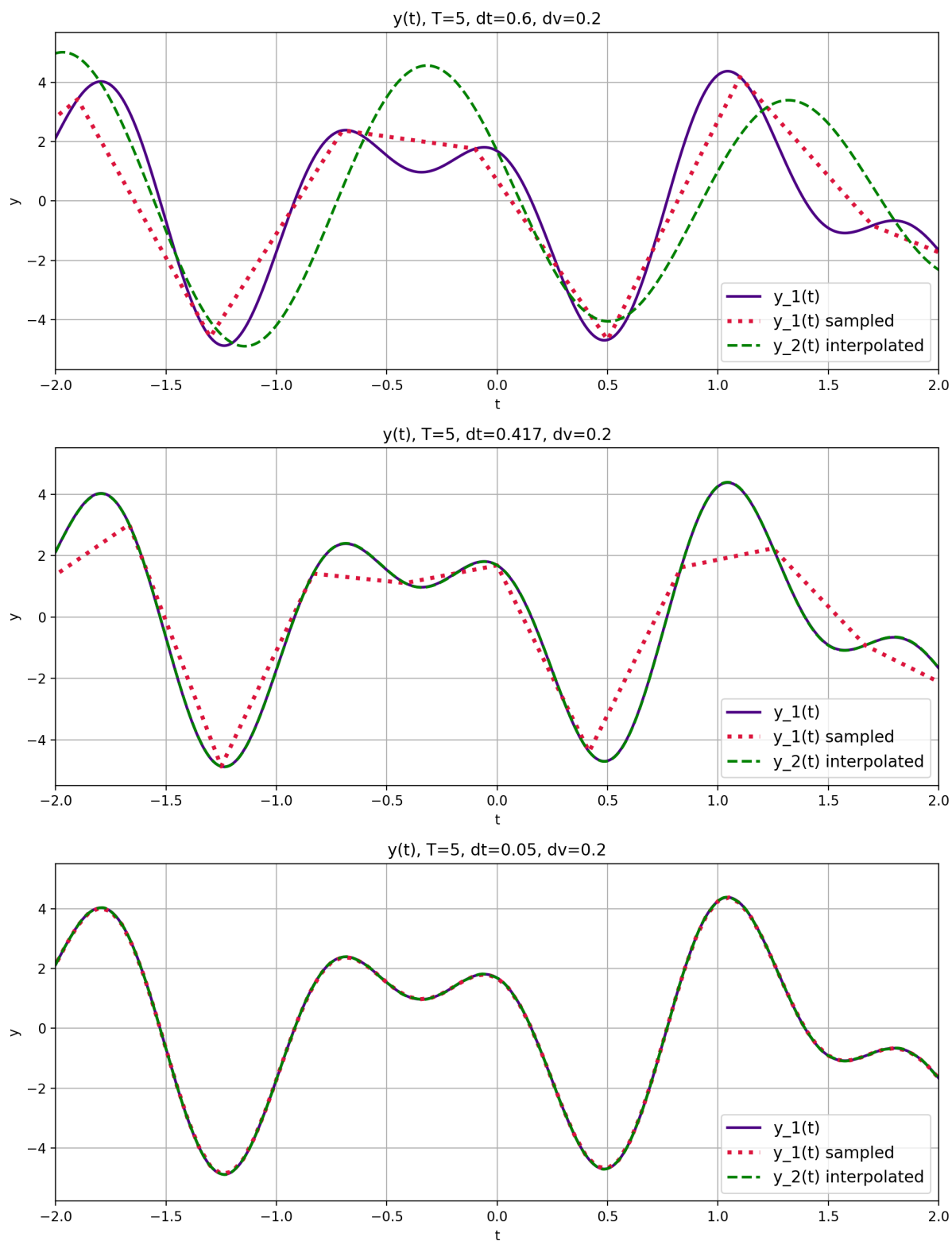


Рисунок 2.4. Сравнение непрерывного, сэмплированного варианта $y_1(t)$ и интерполяции

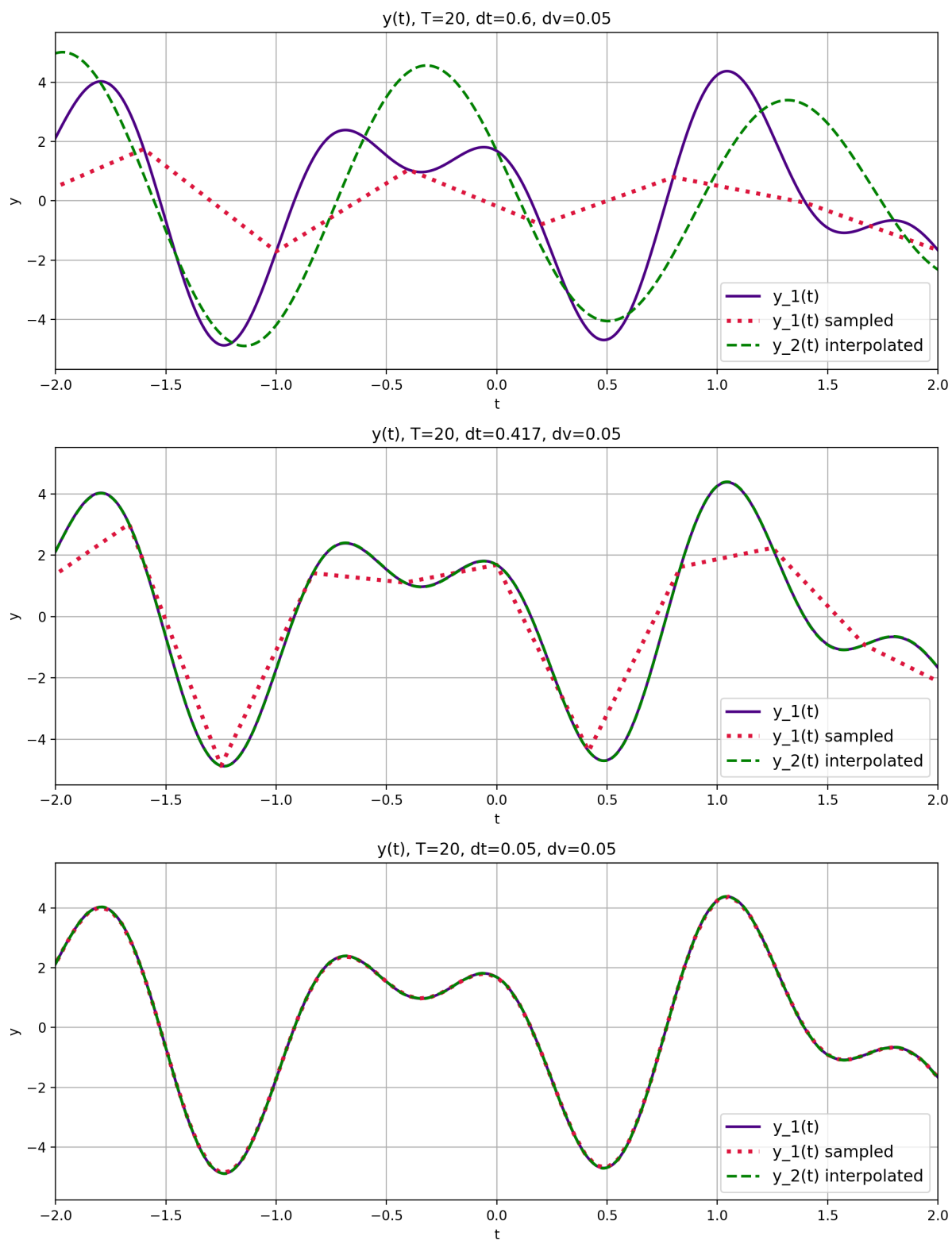


Рисунок 2.5. Сравнение непрерывного и сэмплированного варианта $y_1(t)$ и интерполяции

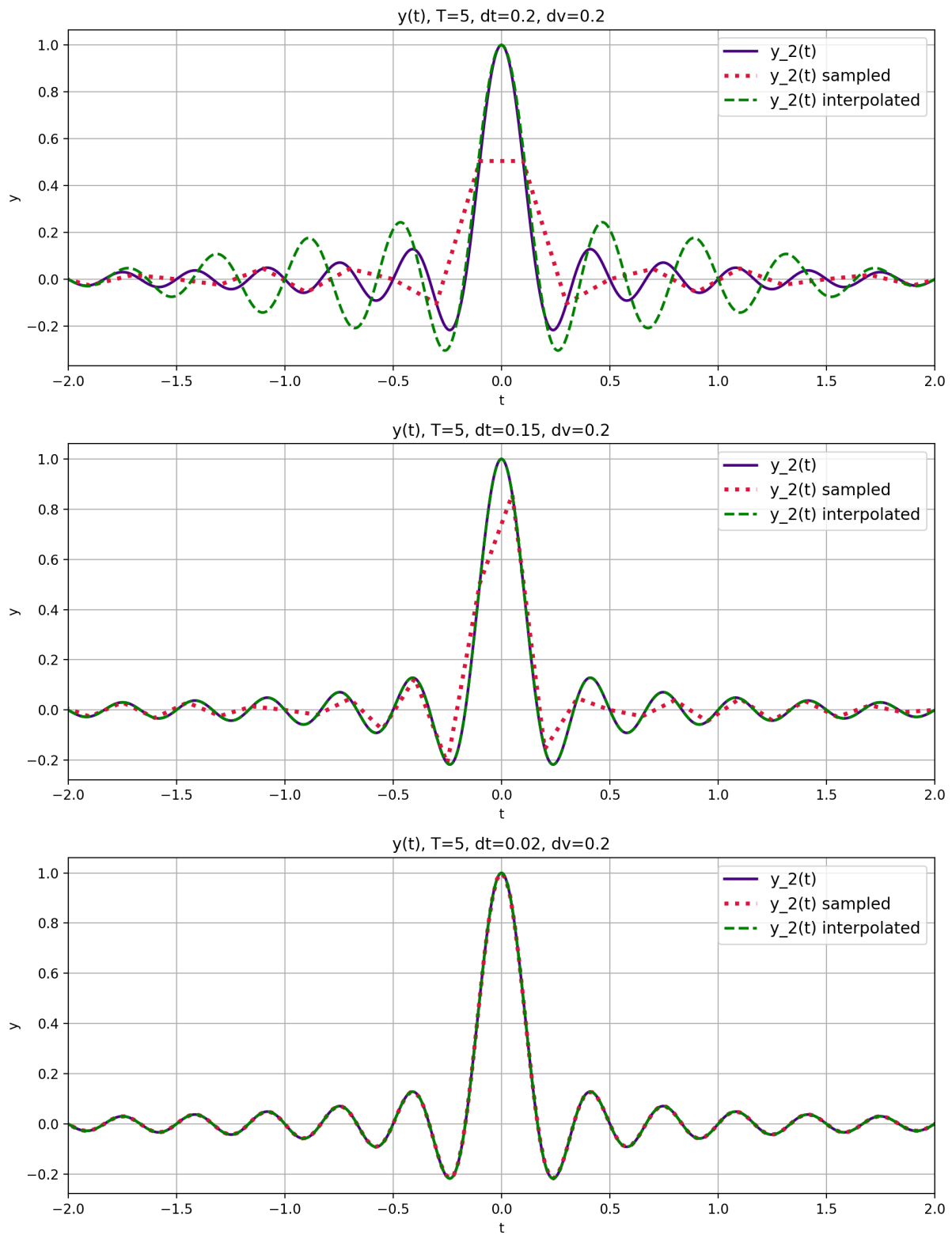


Рисунок 2.6. Сравнение непрерывного, сэмплированного варианта $y_2(t)$ и интерполяции

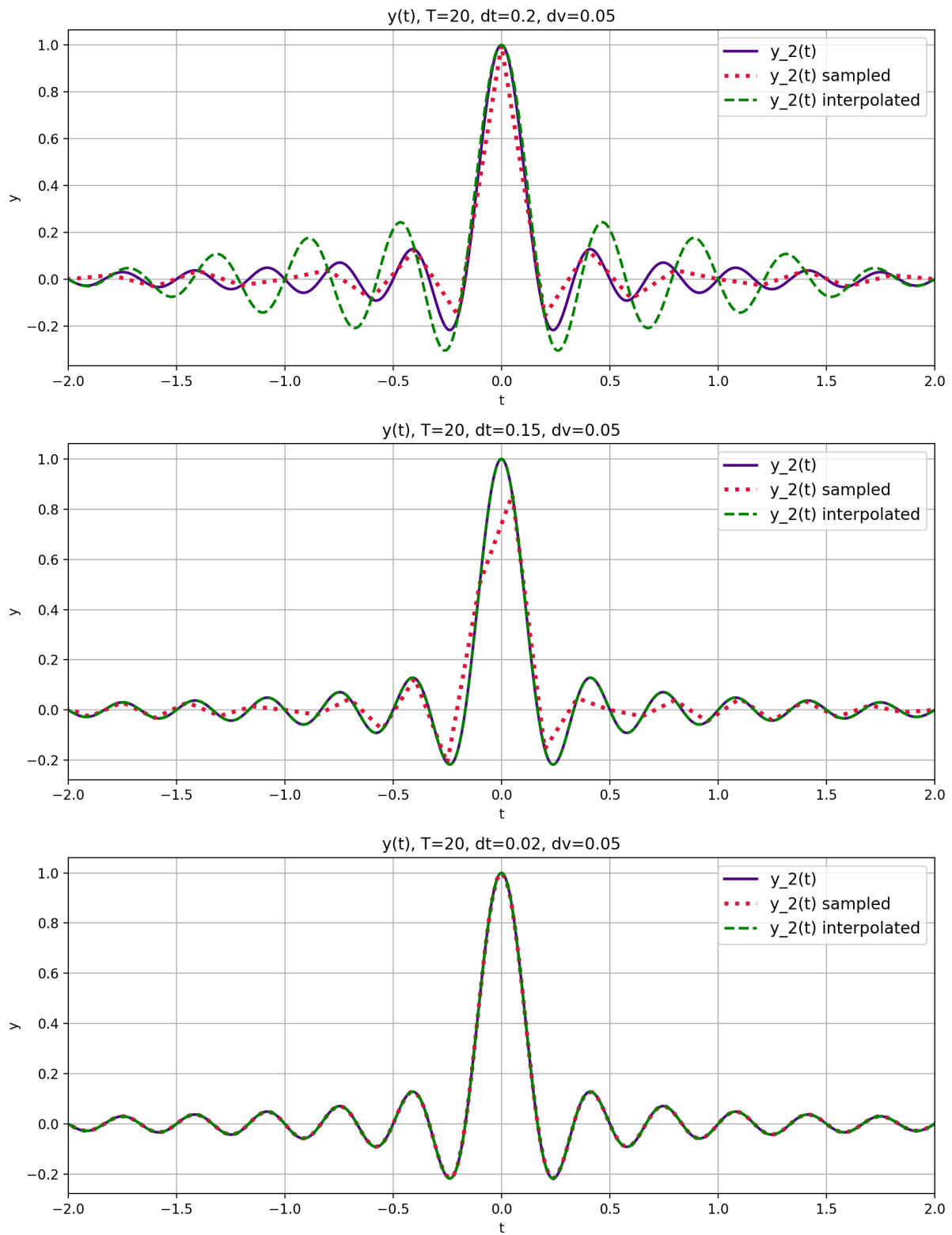


Рисунок 2.7. Сравнение непрерывного и сэмплированного варианта $y_2(t)$ и интерполяции

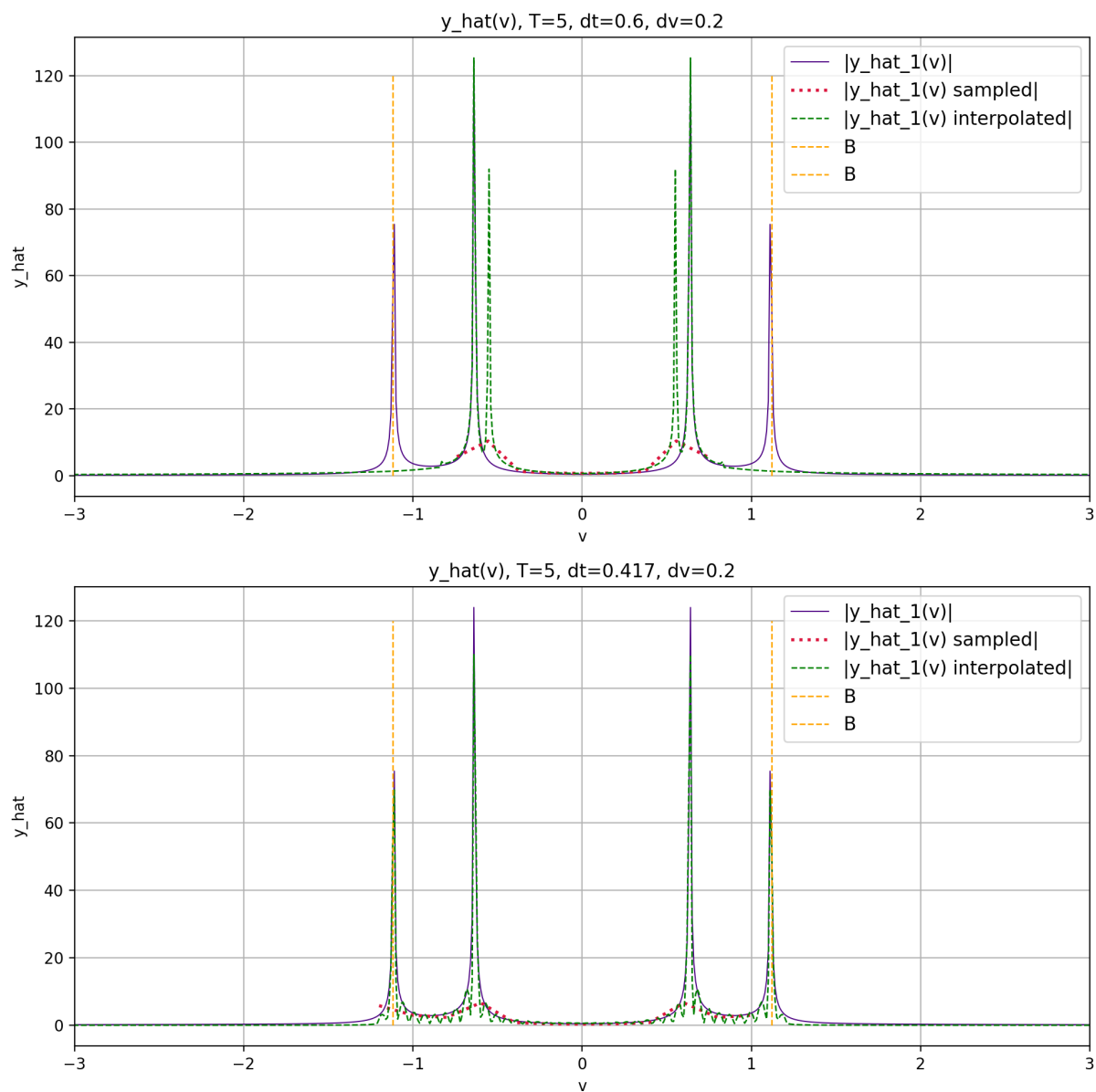


Рисунок 2.8. Сравнение непрерывного, сэмплированного варианта $\hat{y}_1(t)$ и интерполяции

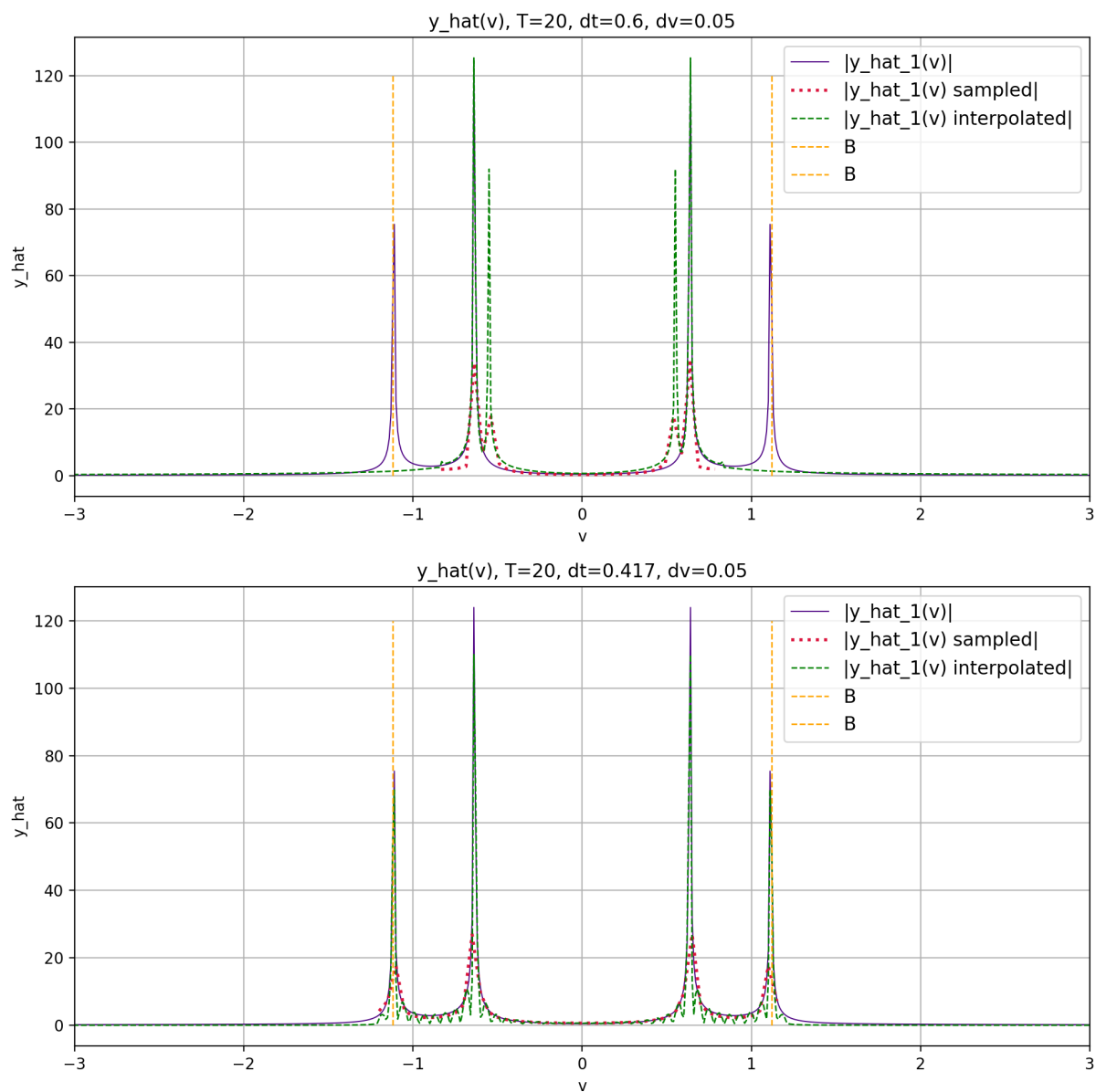


Рисунок 2.9. Сравнение непрерывного и сэмплированного варианта $\hat{y}_1(t)$ и интерполяции

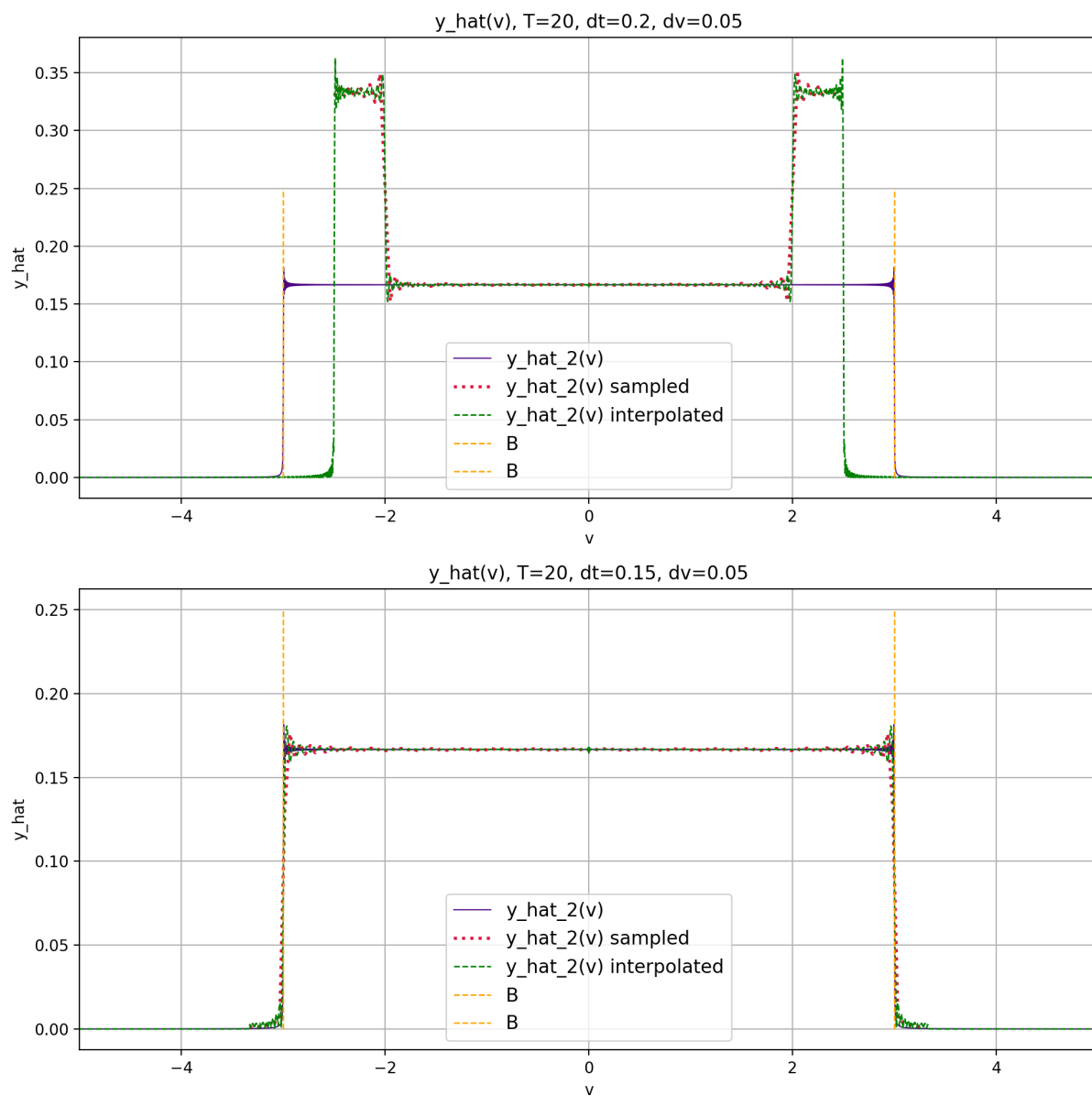


Рисунок 2.11. Сравнение непрерывного и сэмплированного варианта $\hat{y}_2(t)$ и интерполяции

3. Приложение

```
1  # %%
2  import matplotlib.pyplot as plt
3  import numpy as np
4  from numpy.fft import fft, fftshift, ifft, ifftshift, fftfreq
5  from dataclasses import dataclass
6  np.random.seed(17)
7
8  # %% [markdown]
9  # Util-функции
10
11 # %%
12 @dataclass
13 class Plot:
14     x: list
15     y: list
16     label: str = None
17     color: str = None
18     linewidth: float = 1.0
19     linestyle: str = "-"
20     marker: str = None
21     markersize: float = None
22     markerevery: float = None
23
24 class Plot_Group:
25     def __init__(
26         self, title: str | None = None, x_label: str | None = None,
27         ↪ y_label: str | None = None,
28         legend: bool = False, legend_loc: str | None = "best",
29         ↪ legend_fontsize: str | None = "medium",
30         grid: bool = True, xlim: tuple | None = None, ylim: tuple |
31         ↪ None = None
32     ):
33         self.title = title
34         self.x_label = x_label
35         self.y_label = y_label
36         self.legend = legend
37         self.legend_loc = legend_loc
38         self.legend_fontsize = legend_fontsize
39         self.grid = grid
40         self.xlim = xlim
41         self.ylim = ylim
42         self.plots = list()
43
44     def add_plot(self, Plot):
45         self.plots.append(Plot)
46
47 class Chart:
48     def __init__(self, rows: int = 1, cols: int = 1, width: float = 7,
49     ↪ height: float = 5):
50         self.rows = rows
51         self.cols = cols
52         self.width = width
53         self.height = height
54         self.__cur_ij = [0, 0]
55         self.__plot_groups = [[None for _ in range(self.cols)] for _ in
56         ↪ range(self.rows)]
57
58     def add_plot_group(self, Plot_Group: Plot_Group, ij: list = None):
59         if ij is None:
60             if self.__cur_ij[0] ≥ self.rows:
61                 raise IndexError("Max quantity of plots achieved!")
62             ij = self.__cur_ij
63         while self.__plot_groups[self.__cur_ij[0]][self.__cur_ij[1]] is
64         ↪ not None:
65             self.__cur_ij[1] += 1
66             if self.__cur_ij[1] ≥ self.cols:
67                 self.__cur_ij[1] = 0
68                 self.__cur_ij[0] += 1
```

```

63     self.__plot_groups[ij[0]][ij[1]] = Plot_Group
64
65 def __make_chart(self):
66     fig, axes = plt.subplots(self.rows, self.cols,
67                             ↪ figsize=(self.width, self.height))
68     if self.rows == 1 and self.cols == 1:
69         axes = [[axes]]
70     elif self.rows == 1:
71         axes = [axes]
72     elif self.cols == 1:
73         axes = [[ax] for ax in axes]
74
75     for i in range(self.rows):
76         for j in range(self.cols):
77             plot_group_ij = self.__plot_groups[i][j]
78             if plot_group_ij is None:
79                 fig.delaxes(axes[i][j])
80                 continue
81             if plot_group_ij.title:
82                 axes[i][j].set_title(plot_group_ij.title)
83             if plot_group_ij.x_label:
84                 axes[i][j].set_xlabel(plot_group_ij.x_label)
85             if plot_group_ij.y_label:
86                 axes[i][j].set_ylabel(plot_group_ij.y_label)
87             for plot in plot_group_ij.plots:
88                 axes[i][j].plot(
89                     plot.x, plot.y,
90                     label=plot.label,
91                     color=plot.color,
92                     linewidth=plot.linewidth,
93                     linestyle=plot.linestyle,
94                     marker=plot.marker,
95                     markersize=plot.markersize,
96                     markevery=plot.markerevery
97                 )
98             if plot_group_ij.legend and any(plot.label for plot in
99             ↪ plot_group_ij.plots):
100                 axes[i][j].legend(loc=plot_group_ij.legend_loc,
101                 ↪ fontsize=plot_group_ij.legend_fontsize)
102             if plot_group_ij.xlim is not None:
103                 axes[i][j].set_xlim(plot_group_ij.xlim[0],
104                 ↪ plot_group_ij.xlim[1])
105             if plot_group_ij.ylim is not None:
106                 axes[i][j].set_ylim(plot_group_ij.ylim[0],
107                 ↪ plot_group_ij.ylim[1])
108             if plot_group_ij.grid: axes[i][j].grid(True)
109             else: axes[i][j].grid(False)
110
111     plt.gca().set_axisbelow(True)
112     plt.tight_layout()
113
114 def draw_chart(self):
115     self.__make_chart()
116     plt.show()
117
118 def save_chart(self, folder_path:str = "", filename:str = "chart",
119 ↪ dpi="figure"):
120     self.__make_chart()
121     plt.savefig(folder_path+filename+".png", dpi=dpi)
122
123 # %%
124 def list_func(func):
125     def wrapper(x_lst):
126         result = []
127         for x in x_lst:
128             result.append(func(x))
129         return result
130     return wrapper

```

```

129 # %%
130 plt_folder = "plots/"
131
132 # %% [markdown]
133 # # Задание 1
134
135 # %%
136 @list_func
137 def rect_func(t):
138     if np.abs(t) ≤ 0.5: return 1
139     else: return 0
140
141 @list_func
142 def sinc(v):
143     return np.sin(np.pi * v) / (np.pi * v)
144
145 t_rect = np.linspace(-2.0, 2.0, 1000)
146 v_rect = np.linspace(-20.0, 20.0, 1000)
147
148 PI = rect_func(t_rect)
149 PI_hat = sinc(v_rect)
150
151 plt_g = Plot_Group(x_label="t", y_label="Π(t)")
152 plt_g.add_plot(Plot(t_rect, PI, linewidth=2))
153 chart = Chart(width=10)
154 chart.add_plot_group(plt_g)
155 chart.save_chart(plt_folder, "Π(t)", dpi=200)
156 del plt_g, chart
157
158 plt_g = Plot_Group(x_label="v", y_label="Π_hat(v)")
159 plt_g.add_plot(Plot(v_rect, PI_hat, linewidth=2))
160 chart = Chart(width=10)
161 chart.add_plot_group(plt_g)
162 chart.save_chart(plt_folder, "Π_hat(v)", dpi=200)
163 del plt_g, chart
164
165 # %% [markdown]
166 # # Задание 1.1
167
168 # %%
169 t_rect = np.arange(-10.0, 10.0001, 0.0001)
170 v_rect = np.arange(-25.0, 25.0001, 0.0001)
171
172 PI = rect_func(t_rect)
173 PI_hat = sinc(v_rect)
174
175 # %%
176 def point_ft(func, t, v):
177     return func(t) * np.exp(-2j * np.pi * v * t)
178 def point_ifv(func, v, t):
179     return func(v) * np.exp(2j * np.pi * v * t)
180
181 T_list = [2, 8]
182 dt_list = [0.05, 0.001]
183
184 chart_V = Chart(4, 1, 13, 20)
185 chart_T = Chart(4, 1, 13, 20)
186
187 for T in T_list:
188     for dt in dt_list:
189         V = 50
190         dv = 0.25
191         t_space = np.arange(-T/2, T/2 + dt, dt)
192         v_space = np.arange(-V/2, V/2 + dv, dv)
193         PI_hat_trapz = [np.trapz(point_ft(rect_func, t_space, v),
194                                ↪ t_space) for v in v_space]
195         @list_func
196         def PI_hat_trapz_f(v):
197             global v_space
198             return PI_hat_trapz[np.where(v_space==v)[0][0]]
199         PI_trapz = [np.trapz(point_ifv(PI_hat_trapz_f, v_space, t),
200                                ↪ v_space) for t in t_space]

```

```

200 plt_g_V = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={dv}",
    ↪ x_label="v", y_label="Πhat(v)", legend=True,
    ↪ legend_fontsize="large")
201 plt_g_T = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={dv}",
    ↪ x_label="t", y_label="Π(t)", xlim=(-T/2, T/2), legend=True,
    ↪ legend_fontsize="large")
202 plt_g_V.add_plot(Plot(v_rect, PI_hat,
    ↪ "Аналитический образ Πhat(v)", "green", linewidth=2))
203 plt_g_V.add_plot(Plot(v_space, PI_hat_trapz,
    ↪ "Πhat(v) (метод trapz)", "red", linewidth=2,
    ↪ linestyle='--'))
204 plt_g_T.add_plot(Plot(t_rect, PI, "Аналитический образ Π(t)",
    ↪ "indigo", linewidth=2))
205 plt_g_T.add_plot(Plot(t_space, PI_trapz, "Π(t) (метод trapz)",
    ↪ "orangered", linewidth=2, linestyle='--'))
206
207 chart_V.add_plot_group(plt_g_V)
208 chart_T.add_plot_group(plt_g_T)
209 del plt_g_V, plt_g_T
210
211 chart_V.save_chart(plt_folder, "Πhat(v)_comparison_fix_v", dpi=200)
212 chart_T.save_chart(plt_folder, "Π(t)_comparison_fix_v", dpi=200)
213
214 del chart_V, chart_T
215
216 # %%
217 V_list = [8, 40]
218 dv_list = [0.5, 0.001]
219
220 chart_V = Chart(4, 1, 13, 20)
221 chart_T = Chart(4, 1, 13, 20)
222
223 for V in V_list:
224     for dv in dv_list:
225         T = 2
226         dt = 0.01
227         t_space = np.arange(-T/2, T/2 + dt, dt)
228         v_space = np.arange(-V/2, V/2 + dv, dv)
229         PI_hat_trapz = [np.trapz(point_ft(rect_func, t_space, v),
    ↪ t_space) for v in v_space]
230         @list_func
231         def PI_hat_trapz_f(v):
232             global v_space
233             return PI_hat_trapz[np.where(v_space==v)[0][0]]
234         PI_trapz = [np.trapz(point_if(PI_hat_trapz_f, v_space, t),
    ↪ v_space) for t in t_space]
235
236 plt_g_V = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={dv}",
    ↪ x_label="v", y_label="Πhat(v)", xlim=(-V/2, V/2),
    ↪ legend=True, legend_fontsize="large")
237 plt_g_T = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={dv}",
    ↪ x_label="t", y_label="Π(t)", xlim=(-1, 1), legend=True,
    ↪ legend_fontsize="large")
238 plt_g_V.add_plot(Plot(v_rect, PI_hat,
    ↪ "Аналитический образ Πhat(v)", "green", linewidth=2))
239 plt_g_V.add_plot(Plot(v_space, PI_hat_trapz,
    ↪ "Πhat(v) (метод trapz)", "red", linewidth=2,
    ↪ linestyle='--'))
240 plt_g_T.add_plot(Plot(t_rect, PI, "Аналитический образ Π(t)",
    ↪ "indigo", linewidth=2))
241 plt_g_T.add_plot(Plot(t_space, PI_trapz, "Π(t) (метод trapz)",
    ↪ "orangered", linewidth=2, linestyle='--'))
242
243 chart_V.add_plot_group(plt_g_V)
244 chart_T.add_plot_group(plt_g_T)
245 del plt_g_V, plt_g_T
246
247 chart_V.save_chart(plt_folder, "Πhat(v)_comparison_fix_t", dpi=200)
248 chart_T.save_chart(plt_folder, "Π(t)_comparison_fix_t", dpi=200)
249

```

```

250 del chart_V, chart_T
251
252 # %% [markdown]
253 # # Задание 1.2
254
255 # %%
256 T_list = [10, 30]
257 dt_list = [0.1, 0.001]
258
259 chart_V = Chart(4, 1, 13, 20)
260 chart_T = Chart(4, 1, 13, 20)
261
262 for T in T_list:
263     for dt in dt_list:
264         t_space = np.arange(-T/2, T/2 + dt, dt)
265         N = len(t_space)
266         dv = 1 / (N * dt)
267         V = 1 / dt
268         v_space = fftshift(fftfreq(N, dt))
269
270         PI_1 = rect_func(t_space)
271         PI_hat_fft = fftshift(fft(PI_1, norm='ortho'))
272         PI_fft = ifft(ifftshift(PI_hat_fft), norm='ortho')
273
274         plt_g_V = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={round(dv,
275                               3)}", x_label="v", y_label="Π̂(v)", xlim=(-8, 8),
276                               legend=True, legend_fontsize="large")
277         plt_g_T = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={round(dv,
278                               3)}", x_label="t", y_label="Π̂(t)", xlim=(-3, 3), legend=True,
279                               legend_fontsize="large")
280         plt_g_V.add_plot(Plot(v_rect, PI_hat,
281                               "Аналитический образ Π̂(v)", "green", linewidth=0.5))
282         plt_g_V.add_plot(Plot(v_space, np.real(PI_hat_fft),
283                               "Π̂(v) (метод fft)", "red", linewidth=0.5,
284                               linestyle='--'))
285         plt_g_T.add_plot(Plot(t_rect, PI, "Аналитический образ Π(t)",
286                               "indigo", linewidth=2))
287         plt_g_T.add_plot(Plot(t_space, np.real(PI_fft),
288                               "Π̂(t) (метод ifft)", "orangered", linewidth=2,
289                               linestyle='--'))
290
291         chart_V.add_plot_group(plt_g_V)
292         chart_T.add_plot_group(plt_g_T)
293     del plt_g_V, plt_g_T
294
295 chart_V.save_chart(plt_folder, "Π̂(v)_comparison_fft_fix_v", dpi=200)
296 chart_T.save_chart(plt_folder, "Π̂(t)_comparison_fft_fix_v", dpi=200)
297
298 del chart_V, chart_T
299
300 # %%
301 V_list = [10, 50]
302 dv_list = [0.5, 0.01]
303
304 chart_V = Chart(4, 1, 13, 20)
305 chart_T = Chart(4, 1, 13, 20)
306
307 for V in V_list:
308     for dv in dv_list:
309         v_space = np.arange(-V/2, V/2 + dv, dv)
310         N = len(v_space)
311         dt = 1 / (N * dv)
312         T = 1 / dv
313         t_space = fftshift(fftfreq(N, dv))
314
315         PI_1 = rect_func(t_space)
316         PI_hat_fft = fftshift(fft(PI_1, norm='ortho'))
317         PI_fft = ifft(ifftshift(PI_hat_fft), norm='ortho')
318
319         plt_g_V = Plot_Group(title=f"T={T}, dt={round(dt,
320                               3)}, V={V}, dv={dv}", x_label="v", y_label="Π̂(v)",
321                               xlim=(-8, 8), legend=True, legend_fontsize="large")

```

```

310 plt_g_T = Plot_Group(title=f"T={T}", dt={round(dt,
    ↳ 3)}, V={V}, dv={dv}", x_label="t", y_label="Π(t)", xlim=(-3,
    ↳ 3), legend=True, legend_fontsize="large")
311 plt_g_V.add_plot(Plot(v_rect, PI_hat,
    ↳ "Аналитический образ Π_hat(v)", "green", linewidth=0.5))
312 plt_g_V.add_plot(Plot(v_space, np.real(PI_hat_fft),
    ↳ "Π_hat(v) (метод fft)", "red", linewidth=0.5,
    ↳ linestyle='--'))
313 plt_g_T.add_plot(Plot(t_rect, PI, "Аналитический образ Π(t)",
    ↳ "indigo", linewidth=2))
314 plt_g_T.add_plot(Plot(t_space, np.real(PI_fft),
    ↳ "Π(t) (метод ifft)", "orangered", linewidth=2,
    ↳ linestyle='--'))
315
316 chart_V.add_plot_group(plt_g_V)
317 chart_T.add_plot_group(plt_g_T)
318 del plt_g_V, plt_g_T
319
320 chart_V.save_chart(plt_folder, "Π_hat(v)_comparison_fft_fix_t", dpi=200)
321 chart_T.save_chart(plt_folder, "Π(t)_comparison_fft_fix_t", dpi=200)
322
323 del chart_V, chart_T
324
325 # %% [markdown]
326 # Сравнение  $O(N^2)$  и  $O(N \log(N))$ 
327
328 # %%
329 x = np.linspace(0, 10 ** 2, 1000)
330 N2 = x ** 2
331 NlogN = x * np.log(x)
332 chart = Chart()
333 plt_g = Plot_Group(title="N^2 vs Nlog(N)", x_label="N", y_label="Time",
    ↳ legend=True, legend_fontsize="large")
334 plt_g.add_plot(Plot(x, N2, "N^2", linewidth=2, color="royalblue"))
335 plt_g.add_plot(Plot(x, NlogN, "Nlog(N)", linewidth=2, color="red"))
336 chart.add_plot_group(plt_g)
337 chart.save_chart(plt_folder, "N2_NlogN", dpi=200)
338 del plt_g, chart
339
340 # %% [markdown]
341 # # Задание 1.4
342
343 # %%
344 def smart_fft(func, T, dt):
345     t_space = np.arange(-T/2, T/2, dt)
346     N = len(t_space)
347     v_space = fftshift(fftshift(freq(N, dt)))
348     y_space = func(t_space)
349     c_m = dt * np.exp(1j * np.pi * v_space * T)
350     fft_func = fftshift(c_m * fft(y_space))
351     return fft_func, v_space
352
353 def smart_ifft(y_space, v_space, dt):
354     N = len(v_space)
355     T = N * dt
356     t_space = fftshift(fftshift(freq(N, dt)))
357     c_m = dt * np.exp(1j * np.pi * v_space * T)
358     fft_func = ifft(ifftshift(y_space) / c_m)
359     return fft_func
360
361 # %%
362 T_list = [10, 30]
363 dt_list = [0.1, 0.001]
364
365 chart_V = Chart(4, 1, 13, 20)
366 chart_T = Chart(4, 1, 13, 20)
367
368 for T in T_list:
369     for dt in dt_list:
370         t_space = np.arange(-T/2, T/2, dt)
371         N = len(t_space)
372         dv = 1 / (N * dt)

```

```

373 V = 1 / dt
374
375 PI_1 = rect_func(t_space)
376 PI_hat_sfft, v_space = smart_fft(rect_func, T, dt)
377 PI_sfft = smart_ifft(PI_hat_sfft, v_space, dt)
378
379 plt_g_V = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={round(dv,
    ↪ 3)}", x_label="v", y_label="Π̂(v)", xlim=(-8, 8),
    ↪ legend=True, legend_fontsize="large")
380 plt_g_T = Plot_Group(title=f"T={T}, dt={dt}, V={V}, dv={round(dv,
    ↪ 3)}", x_label="t", y_label="Π̂(t)", xlim=(-3, 3), legend=True,
    ↪ legend_fontsize="large")
381 plt_g_V.add_plot(Plot(v_rect, PI_hat,
    ↪ "Аналитический образ Π̂(v)", "green", linewidth=2))
382 plt_g_V.add_plot(Plot(v_space, np.real(PI_hat_sfft),
    ↪ "Π̂(v) (умный fft)", "red", linewidth=2, linestyle='--'))
383 plt_g_T.add_plot(Plot(t_rect, PI, "Аналитический образ Π̂(t)",
    ↪ "indigo", linewidth=2))
384 plt_g_T.add_plot(Plot(t_space, np.real(PI_sfft),
    ↪ "Π̂(t) (умный ifft)", "orangered", linewidth=2,
    ↪ linestyle='--'))
385
386 chart_V.add_plot_group(plt_g_V)
387 chart_T.add_plot_group(plt_g_T)
388 del plt_g_V, plt_g_T
389
390 chart_V.save_chart(plt_folder, "Π̂(v)_comparison_sfft", dpi=200)
391 chart_T.save_chart(plt_folder, "Π̂(t)_comparison_sfft", dpi=200)
392
393 del chart_V, chart_T
394
395 # %% [markdown]
396 # # Сравнение методов
397
398 # %%
399 T = 10
400 dt = 0.01
401
402 chart_V = Chart(1, 1, 10, 6)
403 chart_T = Chart(1, 1, 10, 6)
404
405 t_space = np.arange(-T/2, T/2, dt)
406 N = len(t_space)
407 dv = 1 / (N * dt)
408 V = 1 / dt
409
410 PI_1 = rect_func(t_space)
411 PI_hat_sfft, v_space = smart_fft(rect_func, T, dt)
412 PI_sfft = smart_ifft(PI_hat_sfft, v_space, dt)
413
414 v_space_trapz = np.arange(-20, 20.1, 0.1)
415 PI_hat_trapz = [np.trapz(point_ft(rect_func, t_space, v), t_space) for v
    ↪ in v_space_trapz]
416 @list_func
417 def PI_hat_trapz_f(v):
418     global v_space_trapz
419     return PI_hat_trapz[np.where(v_space_trapz==v)[0][0]]
420 PI_trapz = [np.trapz(point_ifft(PI_hat_trapz_f, v_space_trapz, t),
    ↪ v_space_trapz) for t in t_space]
421
422 PI_hat_fft = fftshift(fft(PI_1, norm='ortho'))
423 PI_fft = ifft(ifftshift(PI_hat_fft), norm='ortho')
424
425 plt_g_V = Plot_Group(title=f"Сравнение всех методов", x_label="v",
    ↪ y_label="Π̂(v)", xlim=(-8, 8), legend=True,
    ↪ legend_fontsize="large")
426 plt_g_T = Plot_Group(title=f"Сравнение всех методов", x_label="t",
    ↪ y_label="Π̂(t)", xlim=(-1.5, 1.5), legend=True,
    ↪ legend_fontsize="large")
427
428 plt_g_V.add_plot(Plot(v_space, np.real(PI_hat_fft),
    ↪ "Π̂(v) (метод fft)", "red", linewidth=1, linestyle='-'))

```

```

429 plt_g_V.add_plot(Plot(v_rect, PI_hat, "Аналитический образ  $\Pi_{\text{hat}}(v)$ ",
    ↪ "blue", linewidth=3, linestyle='-'))
430 plt_g_V.add_plot(Plot(v_space, np.real(PI_hat_sfft),
    ↪ " $\Pi_{\text{hat}}(v)$  (умный fft)", "green", linewidth=3, linestyle='--'))
431 plt_g_V.add_plot(Plot(v_space_trapz, PI_hat_trapz,
    ↪ " $\Pi_{\text{hat}}(v)$  (метод trapz)", "cyan", linewidth=3, linestyle=':'))
432
433 plt_g_T.add_plot(Plot(t_rect, PI, "Аналитический образ  $\Pi(t)$ ", "indigo",
    ↪ linewidth=3))
434 plt_g_T.add_plot(Plot(t_space, np.real(PI_sfft), " $\Pi(t)$  (умный ifft)",
    ↪ "orangered", linewidth=3, linestyle='--'))
435 plt_g_T.add_plot(Plot(t_space, PI_trapz, " $\Pi(t)$  (метод trapz)",
    ↪ "darkgreen", linewidth=2, linestyle='-'))
436 plt_g_T.add_plot(Plot(t_space, np.real(PI_fft), " $\Pi(t)$  (метод ifft)",
    ↪ "orange", linewidth=3, linestyle=':'))
437
438 chart_V.add_plot_group(plt_g_V)
439 chart_T.add_plot_group(plt_g_T)
440 del plt_g_V, plt_g_T
441
442 chart_V.save_chart(plt_folder, " $\Pi_{\text{hat}}(v)$ _comparison_all", dpi=200)
443 chart_T.save_chart(plt_folder, " $\Pi(t)$ _comparison_all", dpi=200)
444
445 del chart_V, chart_T
446
447 # %% [markdown]
448 # # Задание 2
449
450 # %%
451 @list_func
452 def y1(t):
453     return 3 * np.sin(4 * t + np.pi) + 2 * np.sin(7 * t + 1)
454
455 @list_func
456 def y2(t):
457     arg = 6 * t
458     return np.sinc(arg)
459
460 # %% [markdown]
461 # ##### Функция  $y_1$ 
462
463 # %%
464 x = np.linspace(-100.0, 100.0, 10**5)
465 y1_cont = y1(x)
466 y2_cont = y2(x)
467
468 chart = Chart(2, 1, 10, 10)
469 plt_g = Plot_Group("y1(t)", "t", "y", xlim=(-5, 5))
470 plt_g.add_plot(Plot(x, y1_cont, "y1(t)"))
471 chart.add_plot_group(plt_g)
472 del plt_g
473 plt_g = Plot_Group("y2(t)", "t", "y", xlim=(-5, 5))
474 plt_g.add_plot(Plot(x, y2_cont, "y2(t)"))
475 chart.add_plot_group(plt_g)
476 chart.save_chart(plt_folder, "y1_cont", dpi=200)
477 del plt_g, chart
478
479 # %%
480 dt_list = [0.25, 0.15, 0.05]
481
482 chart = Chart(3, 1, 10, 13)
483
484 T = 10
485 for dt in dt_list:
486     x_sampled = np.arange(-T/2, T/2, dt)
487     y1_disc = y1(x_sampled)
488     plt_g = Plot_Group(f"y(t), T={T}, dt={round(dt, 3)}, dv={round(1/T,
    ↪ 3)}", "t", "y", xlim=(-2, 2), legend=True,
    ↪ legend_fontsize="large")
489     plt_g.add_plot(Plot(x, y1_cont, "y1(t)", color="indigo",
    ↪ linewidth=2))
490     plt_g.add_plot(Plot(x_sampled, y1_disc, "y1(t) sampled",
    ↪ color="crimson",

```

```

491         linewidth=2, linestyle="--", marker=".",
           ↪ markersize=7))
492     chart.add_plot_group(plt_g)
493     del plt_g
494
495
496     chart.save_chart(plt_folder, "y1_comparison", dpi=200)
497     del chart
498
499     dt_list = [0.2, 0.1, 0.02]
500
501     chart = Chart(3, 1, 10, 13)
502     for dt in dt_list:
503         x_sampled = np.arange(-T/2, T/2, dt)
504         y2_disc = y2(x_sampled)
505         plt_g = Plot_Group(f"y(t), T={T}, dt={round(dt, 3)}, dv={round(1/T,
           ↪ 3)}", "t", "y", xlim=(-2, 2), legend=True,
           ↪ legend_fontsize="large")
506         plt_g.add_plot(Plot(x, y2_cont, "y_2(t)", color="indigo",
           ↪ linewidth=2))
507         plt_g.add_plot(Plot(x_sampled, y2_disc, "y_2(t) sampled",
           ↪ color="crimson",
           ↪ linewidth=2, linestyle="--", marker=".",
           ↪ markersize=7))
508         chart.add_plot_group(plt_g)
509     del plt_g
510
511
512
513     chart.save_chart(plt_folder, "y2_comparison", dpi=200)
514     del chart
515
516     # %%
517     def interpolate(func, dt, t):
518         f_interpolated = np.zeros(len(t))
519         for n in range(-100, 101):
520             tn = n * dt
521             f_interpolated += func([tn]) * np.sin(np.pi / dt * (t - tn)) /
           ↪ (np.pi / dt * (t - tn))
522         return f_interpolated
523
524     B1 = 1.2
525     B2 = 3
526     T_list = [5, 20]
527
528     dt_list = [0.6, 1 / 2 / B1, 0.05]
529     y1_cont = y1(x)
530     y2_cont = y2(x)
531
532     for T in T_list:
533         chart = Chart(3, 1, 10, 13)
534         for dt in dt_list:
535             x_sampled = np.arange(-T/2, T/2, dt)
536             y1_disc = y1(x_sampled)
537             y1_interp = interpolate(y1, dt, x)
538             plt_g = Plot_Group(f"y(t), T={T}, dt={round(dt,
           ↪ 3)}, dv={round(1/T, 3)}", "t", "y", xlim=(-2, 2),
           ↪ legend=True, legend_fontsize="large")
539             plt_g.add_plot(Plot(x, y1_cont, "y_1(t)", color="indigo",
           ↪ linewidth=2))
540             plt_g.add_plot(Plot(x_sampled, y1_disc, "y_1(t) sampled",
           ↪ color="crimson",
           ↪ linewidth=3, linestyle=":"))
541             plt_g.add_plot(Plot(x, y1_interp, "y_2(t) interpolated",
           ↪ color="green",
           ↪ linewidth=2, linestyle="--"))
542             chart.add_plot_group(plt_g)
543         del plt_g
544
545         chart.save_chart(plt_folder, f"y1_interp_{T}", dpi=200)
546     del chart
547
548     dt_list = [0.2, 0.15, 0.02]
549
550

```

```

551 for T in T_list:
552     chart = Chart(3, 1, 10, 13)
553     for dt in dt_list:
554         x_sampled = np.arange(-T/2, T/2, dt)
555         y2_disc = y2(x_sampled)
556         y2_interp = interpolate(y2, dt, x)
557         plt_g = Plot_Group(f"y(t), T={T}, dt={round(dt,
                    3)}, dv={round(1/T, 3)}", "t", "y", xlim=(-2, 2),
                    ↪ legend=True, legend_fontsize="large")
558         plt_g.add_plot(Plot(x, y2_cont, "y_2(t)", color="indigo",
                    ↪ linewidth=2))
559         plt_g.add_plot(Plot(x_sampled, y2_disc, "y_2(t) sampled",
                    ↪ color="crimson",
                    linewidth=3, linestyle=":"))
560         plt_g.add_plot(Plot(x, y2_interp, "y_2(t) interpolated",
                    ↪ color="green",
                    linewidth=2, linestyle="--"))
561         chart.add_plot_group(plt_g)
562         del plt_g
563     chart.save_chart(plt_folder, f"y2_interp_{T}", dpi=200)
564     del chart
565
566 # %%
567 dt_list = [0.6, 1 / 2 / B1]
568 T_list = [5, 20]
569 x = np.arange(-50.0, 50.0, 0.001)
570 for T in T_list:
571     chart = Chart(2, 1, 10, 10)
572     for dt in dt_list:
573         x_sampled = np.arange(-T/2, T/2 + dt, dt)
574         y1_interp = interpolate(y1, dt, x)
575
576         @list_func
577         def y1i(t):
578             idx = np.abs(x - t).argmin()
579             return y1_interp[idx]
580
581         y_hat_1_cont, v_space_cont = smart_fft(y1, 100, 0.001)
582         y_hat_1_interp, v_space_interp = smart_fft(y1i, 100, 0.001)
583         y_hat_1_disc, v_space_ds = smart_fft(y1, T, dt)
584
585         plt_g = Plot_Group(f"y_hat(v), T={T}, dt={round(dt,
                    3)}, dv={round(1/T, 3)}", "v", "y_hat", xlim=(-3, 3),
                    ↪ legend=True, legend_fontsize="large")
586         plt_g.add_plot(Plot(v_space_cont, np.abs(y_hat_1_cont),
                    ↪ "|y_hat_1(v)|", color="indigo", linewidth=0.75))
587         plt_g.add_plot(Plot(v_space_ds, np.abs(y_hat_1_disc),
                    ↪ "|y_hat_1(v) sampled|", color="crimson",
                    linewidth=2, linestyle=":"))
588         plt_g.add_plot(Plot(v_space_interp, np.abs(y_hat_1_interp),
                    ↪ "|y_hat_1(v) interpolated|", color="green",
                    linewidth=1, linestyle="--"))
589
590         xB = [1.12] * 2
591         yB = [0, 120]
592         plt_g.add_plot(Plot(xB, yB, "B", "orange", linestyle="--"))
593         xBm = [-1.12] * 2
594         yBm = [0, 120]
595         plt_g.add_plot(Plot(xBm, yBm, "B", "orange", linestyle="--"))
596         chart.add_plot_group(plt_g)
597         del plt_g
598
599     chart.save_chart(plt_folder, f"y_hat_1_interp_{T}", dpi=200)
600     del chart
601
602 dt_list = [0.2, 0.15]
603 for T in T_list:
604     chart = Chart(2, 1, 10, 10)
605     for dt in dt_list:
606         x_sampled = np.arange(-T/2, T/2, dt)

```

```

611 y2_interp = interpolate(y2, dt, x)
612
613 @list_func
614 def y2i(t):
615     idx = np.abs(x - t).argmin()
616     return y2_interp[idx]
617
618 y_hat_2_cont, v_space_cont = smart_fft(y2, 200, 0.001)
619 y_hat_2_interp, v_space_interp = smart_fft(y2i, 200, 0.001)
620 y_hat_2_disc, v_space_ds = smart_fft(y2, T, dt)
621
622 plt_g = Plot_Group(f"y_hat(v), T={T}, dt={round(dt,
623     3)}, dv={round(1/T, 3)}", "v", "y_hat", xlim=(-5, 5),
624     ↪ legend=True, legend_fontsize="large")
625 plt_g.add_plot(Plot(v_space_cont, np.abs(y_hat_2_cont),
626     ↪ "y_hat_2(v)", color="indigo", linewidth=0.75))
627 plt_g.add_plot(Plot(v_space_ds, np.abs(y_hat_2_disc),
628     ↪ "y_hat_2(v) sampled", color="crimson",
629         linewidth=2, linestyle=":"))
630 plt_g.add_plot(Plot(v_space_interp, np.abs(y_hat_2_interp),
631     ↪ "y_hat_2(v) interpolated", color="green",
632         linewidth=1, linestyle="--"))
633 xB = [3] * 2
634 yB = [0, 0.25]
635 plt_g.add_plot(Plot(xB, yB, "B", "orange", linestyle="--"))
636 xBm = [-3] * 2
637 yBm = [0, 0.25]
638 plt_g.add_plot(Plot(xBm, yBm, "B", "orange", linestyle="--"))
639 chart.add_plot_group(plt_g)
640 del plt_g
641
642 chart.save_chart(plt_folder, f"y_hat_2_interp_{T}", dpi=200)
643 del chart

```

Листинг 1: Листинг